```
// ===== miniprogram/app.js — 小程序入口 =====
const { request } = require('./utils/api')
const posterTemplates = require('./utils/posterTemplates')
const { themeOf, setActiveTheme, dimAt, dimNext, dimDotStyle, dimScrimStyle } =
require('./utils/palette')
const { t } = require('./utils/i18n')

// 分享卡片的路径上带着邀请人的 user id（?inviter=123）。落到本地存储是因为它必须活到
// "这个人真的写下第一篇笔记"那一刻：冷启、热启、从分享卡片直接落到新建页，都只有这里
// 能一次接住。到底认不认、什么时候给额度，全在服务端判（见 backend/app/services/quota.py）。
const INVITER_KEY = 'inviterId'

// 界面字体同理是本机偏好：它只影响这一台设备上的字长什么样，不需要同步，
// 也不该同步——安卓上这三档基本命不中，跟着账号跑到别人设备上只会让人看到"没生效"。
const UI_FONT_KEY = 'uiFont'
const UI_FONT_CLASS = { song: 'font-song', fang: 'font-fang', kai: 'font-kai' }

// 背景形象图压多深，同样是本机偏好：它只改这一台设备上那张图的明暗，
// 跟着账号跑到另一台设备上，只会让人看到"我的图怎么自己变黑了"。
// 三档的值与那两枚灰度都在 utils/palette.js 的 BG_DIMS 里，这里只管存哪一个档被选中。
const BG_DIM_KEY = 'bgDim'

function rememberInviter(options) {
  const raw = options && options.query ? options.query.inviter : ''
  const id = parseInt(raw, 10)
  if (!id || id <= 0) return
  if (wx.getStorageSync(INVITER_KEY) === id) return
  wx.setStorageSync(INVITER_KEY, id)
}

App({
  globalData: {
    userInfo: null,
    isLoggedIn: false,
    userId: '',
    token: '',
    loginPromise: null,
    // 私密笔记的解锁凭证（{ token, expireAt }）。只存内存：杀了重进就要重新输密码，
    // 这一闸防的正是"手机在别人手里"那一段时间。读写都在 utils/api.js 里收口。
    privateUnlock: null,
  },

  onLaunch(options) {
    rememberInviter(options)
    // 字体那一排入口已撤（iOS 与安卓真机都不换字），但测试期可能在这台机器上留下过选择。
    // 留着它就等于一个看不见的开关在生效，所以启动先清掉；重新做字体时删掉这一行即可。
    wx.removeStorageSync(UI_FONT_KEY)
    // 卡片模板配方：先把上一回拉到的那批端回来（只读本机存储、同步、不查网），
    // 于是这一次会话从第一张卡起吃的就是上次那批下发值。拉新的一批在「卡片模板」页里。
    // 坏了也只是"这批没有"——包内那十套是 poster.js 里的常量，跟这里通不通没关系。
    posterTemplates.restore()

    this.login()
  },

  // 小程序已经活着的时候点开分享卡片只补发 onShow，不会走 onLaunch。
  onShow(options) {
    rememberInviter(options)
    // 冷启由 _doLogin 带着 inviter 一起走；这里只管"已经登录着"的热启那条路。
    if (this.globalData.isLoggedIn && wx.getStorageSync(INVITER_KEY)) {
      this._reportInviter()
    }
  },
```

```
async _reportInviter() {
  const inviter = wx.getStorageSync(INVITER_KEY)
  if (!inviter) return
  try {
    await request('/api/user/inviter', 'POST', { inviter })
    // 认不认都由服务端判，而且判过就不会再改（已有归属/名下已有笔记都挡着），
    // 所以只要这一趟通了就把本地这份扔掉——留着它每次启动都要白跑一次请求。
    wx.removeStorageSync(INVITER_KEY)
  } catch (err) {
    // 网络这一类失败要留着下次再报，不然这次归因就永久丢了。
    console.error('邀请归因上报失败，保留待下次重试:', err)
  }
},

getLoginPromise() {
  // 已经登录过就直接给一个已完成的 Promise。原来这里在登录成功后把
  // loginPromise 置空，于是每次 onShow 等它都会重新走一遍 wx.login + 换票——
  // 新建页当上启动页之后，从相机/相册返回也要等一次，纯属白跑。
  if (this.globalData.isLoggedIn && this.globalData.token) {
    return Promise.resolve()
  }
  if (!this.globalData.loginPromise) {
    this.globalData.loginPromise = this._doLogin().finally(() => {
      this.globalData.loginPromise = null
    })
  }
  return this.globalData.loginPromise
},

async _doLogin() {
  try {
    const loginRes = await new Promise((resolve, reject) => {
      wx.login({
        success: resolve,
        fail: reject,
      })
    })
    const body = { code: loginRes.code }

    const inviter = wx.getStorageSync(INVITER_KEY)
    if (inviter) body.inviter = inviter
    const res = await request('/api/auth/wechat', 'POST', body)
    this.globalData.token = res.token
    this.globalData.userId = res.user_id
    this.globalData.userInfo = {
      nickName: res.nickname || '',
      avatarUrl: res.avatar_url || '',
      language: res.language || 'zh',
      wallpaper: this.getWallpaper(res.wallpaper),
    }
    this.globalData.isLoggedIn = true
    this.applyTheme(this.globalData.userInfo.wallpaper)
  } catch (err) {
    console.error('登录失败:', err)
    throw err
  }
},

async login() {
  return this.getLoginPromise()
},

// 注销之后本地这一整套都要跟着清掉：留着旧 token 会让每一个请求都去撞 401，
// 留着 inviter 会让一个已经注销过的号再去成就别人一次。
```

```
  clearSession() {
    this.globalData.token = ''
    this.globalData.userId = ''
    this.globalData.userInfo = null
    this.globalData.isLoggedIn = false
    this.globalData.loginPromise = null
    wx.removeStorageSync(INVITER_KEY)
  },

  /**
   * 当前该用哪套壁纸：本机那份优先，没有才用服务端的。
   * @param fromServer 登录接口返回的原值，只在 _doLogin 那一步传进来
   *
   * 返回的是**解析后的 canonical key**（themeOf 里那张 WALLPAPER_ALIAS 表把六枚旧值压成四枚）。
   * 这一句 10-04 改过两次：
   * ① 原来这里判 `themeOf(wanted).dark ? 'default' : wanted`——深色那两枚（夜紫/深海）09-30 起
   *    整条链路屏蔽，因为这一屏永远铺背景图，而深色主题的卡底是 rgba(255,255,255,.05) 那层薄膜，
   *    贴在照片上等于没有，站长真机反馈"卡片是透明的、完全看不清"。现在四枚壁纸全浅色、
   *    旧 key 一律由别名表落到浅色那枚，那个三元成了死路，所以撤干净，不留兼容壳。
   * ② 原来直接把 wanted 原样返回，于是存过 'default' 的人拿到的就是 'default'，
   *    而壁纸条里已经没有这一格 → 实际生效的是象牙，条上却一格都不亮（外观设置页那个高亮 bug）。
   *    取 .key 就是把这件事收在一处：往下所有人拿到的都是四枚之一的 key。
   */
  getWallpaper(fromServer) {
    const wanted = fromServer || (this.globalData.userInfo && this.globalData.userInfo.wallpaper)
|| 'default'

    return themeOf(wanted).key
  },

  /**
   * 选完壁纸之后落一次账。四枚现在都由服务端当家（站长 10-04 拍「统一」，
   * 后端白名单已加这三个新 key），所以这里只更新内存里那一份，不再写本机存储。
   */
  setWallpaper(key) {
    if (this.globalData.userInfo) this.globalData.userInfo.wallpaper = key
    return key
  },

  uiFont() {
    const k = wx.getStorageSync(UI_FONT_KEY)
    return UI_FONT_CLASS[k] ? k : 'default'
  },

  setUIFont(key) {
    const k = UI_FONT_CLASS[key] ? key : 'default'
    if (k === 'default') wx.removeStorageSync(UI_FONT_KEY)
    else wx.setStorageSync(UI_FONT_KEY, k)
    return k
  },

  uiFontClass() {
    return UI_FONT_CLASS[this.uiFont()] || ''
  },

  /**
   * 背景形象图那一档深浅，三页（新建 / 笔记 / 我的）每次进页读一次。
   * dimV 是当前档（0 最亮 / 1 压一半 / 2 现网那一档），dimDot 是那一枚点的灰度，
   * dimScrim 是**现网那层压暗罩自己**的透明度样式串——不是再叠一层黑。
   * 最沉那一档 dimScrim 是空串（罩子照 CSS 全铺）；最亮那一档是 opacity:0，
   * 照片就等于原图，站长拿原图对过才认的这一档。
   */
  bgSkin() {
    const v = dimAt(wx.getStorageSync(BG_DIM_KEY)).v
```

```
    return { dimV: v, dimDot: dimDotStyle(v), dimScrim: dimScrimStyle(v) }
  },

  // 点一下换一档：最亮 → 压一半 → 全铺 → 回最亮。存完把新那档原样返回，页面 setData 就行。
  cycleBgDim() {
    const v = dimNext(dimAt(wx.getStorageSync(BG_DIM_KEY)).v)
    wx.setStorageSync(BG_DIM_KEY, v)
    return { dimV: v, dimDot: dimDotStyle(v), dimScrim: dimScrimStyle(v) }
  },

  // .container 上那一串类名 = 主题 + 界面字体。页面只管贴，不各自拼第二份规则。
  containerClass(wallpaper) {
    return [themeOf(wallpaper).cls, this.uiFontClass()].filter(Boolean).join(' ')

  },

  getThemeClass(wallpaper) {
    return this.containerClass(wallpaper)
  },

  applyTheme(wallpaper) {
    // 先把当前主题记进 palette：方块按分类取哪一档、新建页那三张卡用什么面，
    // 全看这一步有没有先落地（palette.js 里 ACTIVE_THEME 那段注释写了为什么做成模块状态）。
    const theme = setActiveTheme(wallpaper)
    // 导航条必须和页面底色同值，否则卡片滚到顶部会看出一条色差。
    // 之前这里写死 '#f5f5f5'，和四套主题的底色一个都对不上。
    // 页面刚 onLoad 时这个接口可能直接 fail，忽略即可，底色由容器自己画。
    wx.setNavigationBarColor({
      frontColor: theme.dark ? '#ffffff' : '#000000',
      backgroundColor: theme.page,
      fail() {},
    })
    // tab 栏不在这里推：getTabBar 是 Page 的 API，App 实例上根本没有这个方法，
    // 原来那句 this.getTabBar?.() 恒为 undefined，等于没人管过 tab 栏的颜色和字体。
    // 三个 tab 页各自在 onShow 里已经有 this.getTabBar() 那一串，由它们推。
    return this.containerClass(wallpaper)
  },

  // 铺了形象图的那三页（新建 / 笔记 / 我的），导航条跟着翻成深底白字。
  // 「按背景深浅自动反差」这条为什么不用去读那张图：图上面压的那层罩子顶部是
  // rgba(18,20,26,.58)，不管用户换哪张形象图，压完都是 #181A20 这一档的深，字就得是纸白。
  // 没铺图的态不碰导航条——上面 applyTheme 已经按壁纸底色设过了，别在这里把主题色改丢。
  // 原来这条判断在笔记页和新建页里各写一份，「我的」页漏了，于是三页的导航条两深一浅。
  applyNavForBand(hasBg) {
    if (!hasBg) return
    wx.setNavigationBarColor({
      frontColor: '#ffffff',
      backgroundColor: '#181a20',
      fail() {},
    })
  },

  // 导航条标题原来只写在 pages/*/*.json 里，全是硬编码中文：英文用户在语言页切完，
  // 满屏内容都变了、顶上那行还是中文。JSON 没法动态，所以统一由页面在同步 lang 时调这里。
  setNavTitle(key, lang) {
    const k = lang || this.globalData.userInfo?.language || 'zh'
    wx.setNavigationBarTitle({ title: t(key, k), fail() {} })
  },
})

/* ===== miniprogram/app.wxss — 全局样式 ===== */
/* ------------------------------------------------------------------
   设计令牌（V2 左块右文）。分工：
```

```
   ① 非色彩令牌——全局一份，不随主题变。

   ② 主题（=用户选的壁纸）只管 chrome：页面底、导航、搜索框、chips、正文文字。
   ③ 方块的颜色不归主题管，由 utils/palette.js 按分类固定注入 --blk-bg / --blk-ink。
      10-04 起这一条成了例外中的全部：四枚壁纸都带 ramp（整套色阶），方块一律跟着
      当前主题那一支色相走，所以它们的方块色在下面各主题块与 palette.js 里各存一份镜像值。

   V1→V2 只改了承载方式：颜色从"整张卡"退到卡片左侧那块方块，卡本身回白底。
   这一改带来两个后果，写清楚免得下一个人当成 bug：
   ① 描边（--w-edge + --card-edge）取代投影成为卡片的边界语言，投影令牌已经删干净，
      只有浮层（弹窗、tab 栏）还留着 --sh-float。
   ② 卡不再是饱和色，所以深色壁纸必须自己出一套卡（半透明底 + 亮描边 + 亮字）。
      --bg-card / --card-edge 因此从兜底值升级成主题变量，切壁纸时它们必须跟着变。
   ------------------------------------------------------------------ */

page {
  /* 字阶：八级。最下面两档只给"方块里的日期"和"小眉标"用，正文不下探到这里 */
  --fs-h1: 42rpx;
  --fs-h2: 34rpx;
  --fs-title: 31rpx;
  --fs-body: 28rpx;
  --fs-meta: 24rpx;
  --fs-tiny: 21rpx;
  --fs-label: 20rpx;
  --fs-micro: 18rpx;
  --lh-body: 1.6;
  --lh-tight: 1.38;

  /* 间距模数：8rpx 一档 */
  --sp-1: 8rpx;
  --sp-2: 16rpx;
  --sp-3: 24rpx;
  --sp-4: 32rpx;
  --sp-5: 48rpx;
  --sp-6: 64rpx;
  /* 行卡的三个数（块-文间距、上下留白、左右留白）在效果图里是同一个值，这里也锁成一个 */
  --sp-row: 26rpx;

  /* 圆角与描边——描边是这套方向的骨架，投影已经退役 */
  --r-card: 40rpx;
  --r-block: 32rpx;
  --r-chip: 28rpx;
  --r-pill: 999rpx;
  --r-btn: 16rpx;
  --w-edge: 3rpx;
  --sh-float: 0 16rpx 44rpx rgba(35, 37, 44, 0.22);

  /* 方块：176rpx 见方，母题住在它里面 */
  --blk: 176rpx;
  --blk-pad: 18rpx;
  --fs-block: 32rpx;
  --motif-size: 124rpx;

  --motif-shift: 32rpx;

  /* chrome 色彩兜底：这一份就是象牙那一套（THEMES[0]，未登录/脏值都落在这里）。
     下面四个 .theme-* 块各写一份镜像值，等值由色板那把尺子逐枚钉死。 */
  --text-primary: #352E1D;
  /* 二级字 0.66 → 0.72（站长 10-04 拍）：压在卡底实测象牙/天青只有 4.35/4.38，本来就不过
     正文门槛 4.5；「我的」那行 LOGO / 名称 / Slogan 撤掉自己的框退到页底之后更暗（页底比卡底
     低 ΔL 8 档），同一档只有 3.99~4.34。抬到 .72 是两面同时过线的最低值（.70 还差一点）。 */
  --text-secondary: rgba(53, 46, 29, 0.72);
  /* 三级字 0.4 →（10-04）0.55 →（10-05 真机）0.72。
     10-04 那次抬到 .55 写的理由是"仍比二级字浅一档"——今天他真机（天青）连着两句：
```

```
    "部分数字颜色太浅，与背景太接近，很难看清"、"版本号也很浅"。按四套壁纸实算，.55 压在
    页底只有 **3.03~3.25:1**、卡底 **3.21~3.52:1**：连 WCAG 大字门槛 3.0 都是勉强贴线，
    而这些字全是 20~24rpx 的小字，按小字门槛 4.5 算四套一套都没过。
    抬到 .72 是**四套壁纸、页底与卡底两面都过 4.5 的最低档**（页底 4.70~5.15、卡底 5.19~5.87；
    `探-数字对比度` 在天青真机上量到 4.67/5.17，比这两头略低，差的是合成前景时取整的那半档）。
    再往上一档（.75）本身没问题，但 .72 已经是二级字那一档，三级字比它深就倒过来了。
    代价要说清楚：这一档与 `--text-secondary` 合并成同一个值，"次要"这件事从此只靠**字号与字重**
    表达，不再靠深浅。要保留三档深浅就得二级字再往上抬（.85 起），那是改他 10-04 拍过的值，
    等他点头再动。 */
  --text-tertiary: rgba(53, 46, 29, 0.72);
  --bg-page: #E2DED4;
  --bg-card: #F3F3F1;
  --bg-card-glass: rgba(243, 243, 241, 0.85);
  --accent: #352E1D;
  --accent-soft: rgba(53, 46, 29, 0.07);
  /* 实心按钮那一块面与压在上面的字（站长 10-04：深色按钮必须和底部导航"选中那一格"
     下面那块圆底同色，无论哪套壁纸）。唯一真相是 palette.chromeOf(壁纸).sel / .ink。
     10-04 第二轮：sel 的明度不再写死（原来一律"底栏条抬 16 档"＝L36.5，只看色相不看对比），
     改成按纸白 4.7 反解——于是"两处同色"和站长那句"不想太深"同时成立。字仍是纸白。 */
  --btn-bg: #766948;
  --btn-ink: #F2EFE9;
  --border: rgba(53, 46, 29, 0.09);
  --card-edge: rgba(53, 46, 29, 0.12);
  --chip-idle: rgba(53, 46, 29, 0.07);
  --danger: #b4231f;
  --danger-soft: rgba(180, 35, 31, 0.08);

  background-color: var(--bg-page);
  font-family: -apple-system, BlinkMacSystemFont, 'PingFang SC', 'Helvetica Neue', sans-serif;
  color: var(--text-primary);
}

/* ---------- 全局那一档"数字"：Poppins Thin ----------
   这一支现在管的是**所有纯数字加符号的显示字段**：「我的」右上角那枚大数字、
   首页顶部三列、列表日期 MM-DD、要点序号、字数 137/500、版本号 v1.9.7。
   声明只放在这里（原来在 pages/me/me.wxss），各页只写 font-family、不抄第二份 base64
   ——`验-列表头部铺图.js` 有一条专门数这段声明的出现次数（所以这条注释里也不再提它的名字，
   写了就会被那条尺子当成第二份）。
   站长要的是 iPhone 锁屏时钟那种"细、长、圆"。那版真身是 SF Pro Rounded，

   苹果许可只允许在苹果设备上用、不能打进包分发，所以走开源最接近的一支：
   Poppins Thin 100（OFL，可商用可内嵌）。字体只裁要用的那十几个字符、base64 内嵌，
   2.2KB 进包，不占域名、不会出现"字体还没到、数字先跳一下"。
   字符集站长 10-01 晚扩了一次：从只有 0-9 扩到 `0-9 - / . : + 空格 v`，
   加新字符要重出一版子集，判据写在 `验-数字字体全局.js`（它逐条比对界面用到的符号
   在不在子集里，缺一个就会在真机上掉回系统字体、全局数字风格当场分成两半）。
   字号一条都不动——只换字形，不改大小（他原话："但只改字体，之前字体尺寸不改变"）。 */
@font-face {
  font-family: 'WtsjMind';
  src:
url(data:font/ttfbase64,AAEAAAANAIAAAwBQRFNJRwAAAAEAAAjAAAAACEdQT1NEdkx1AAAIcAAAACBHU1VCH0gnawAA
CJAAAAAwT1MvMljcW0EAAAFYAAAAYGNtYXABDwCJAAACBAAAAExnbHlmm+0G8wAAAngAAARaaGVhZCCu5CUAAADcAAAANmhoZ
WEGjAErAAABFAAAACRobXR4IoICeAAAAbgAAABMbG9jYQiYCdIAAAJQAAAAKG1heHAAhwBoAAABOAAAACBuYW1lHvI2PwAABt
QAAAF6cG9zdP+4ADIAAAhQAAAAIAABAAAABAEGPGSDL18PPPUAAwPoAAAAANikqbIAAAAA5uP1fv/d/2QCLwOnAAAABwACAAA
AAAAAAAAEAAAQa/qIAZAJm/90ADwIvAAEAAAAAAAAAAAAAAAAAAAAATAAEAAAATADAAAwAYAAIAAQACAB4ABgAAAGQAAAACAAEA
BAM7APoABQAAAooCWAAAAEsCigJYAAABXgAyAUQAAAAAAwAAAAAAAAAAAAABAAAAAAAAAAAAAAAAAAASVRGTwDAACAAdgQa/qIAZ
ARvAnMAAAABAAAAAAIcAr4AAAAgAAQB9AAAASIAAAJNACgBvP/dAHQAIAFnAB4CTABGAOIADwI3ADICPwA1Aj4AFAJQAEYCZg
BGAfYACgJgADcCZgA8AHQAIAIDABQBvAAoAAAAAgAAAAMAAAAUAAMAAQAAABQABAA4AAAACgAIAAIAAgAgACsAOgB2//8AAAA
gACsALQB2////4f/X/9b/mwABAAAAAAAAAAAAAAAAAAAAAAAAAVAB0AMwBDAHUAhACqAOMA/gEvAWwBgAHGAgICDgIgAi0AAQAo
AH0CJQJ7AAsAAAEjFSM1IzUzNTMVMwIl9RT09BT1AXP29hL29gD////dAXMBSQGFAAIAErUAAAEAIP/+AFQANAALAAAWJjU0N
jMyFhUUBiMvDw8LCw8PCwIPDAwPDwwMDwAAAQAe/2QBSQOnAAMAAAEBIwEBSf7pFAEVA6f7vQRDAAACAEb/+QIGAscADwAfAA
ASNjYzMhYWFRQGBiMiJiY1JCYmIyIGBhUUFhYzMjY2NUYkYlpZYyQkY1laYiQBrCBZU1NZICBZU1NZIAHTmlpam3BynFtbnHJ
sklVVkmxtlFZWlG0AAQAPAAAAjQLAAAUAABM1MxEjEQ9+FAKuEv1AAq4AAQAyAAACBQLHABcAADYSNTQmIyIGByM2NjMyFhUU
BgYHIRUhNfXkVGBhZwcUCHdkVXN+ul0Brf4tiQEEdk5kdWBse15mWb6jNxILAAEANf/5AfwCxwAoAAASNjMyFhUUBgcVFhYVF


```

```
AYjIiYnMxYWMzI2NTQmIyM1MzI1NCYjIgYHI0x4ZV5oUl1UaGpqY38RFA9zXV5iinwREvheVFJ2CxQCX2hiTEBfCAYJX1FRaW
thUmZdSVdbE6FGVlZbAAIAFAAAAi8CwAAKAA0AADc1ATMRMxUjFSM1EQEhFAGNFHp6FP6MAXS+EAHy/hASvr4B4v4wAAEARv/
5AhkCwAAfAAABIRE2NjMyFhUUBgYjIiYnMxYWMzI2NTQmIyIGByMRIQH3/m4dcEJ0cTBpUF5+DhQRclJqbG1kRW4eEgGmAq7+
wCs0gWBCbkNqXVdce2Rgbzo0AXMAAgBG//kCKgLIABsAKQAAACYjIgYVFBc2NjMyFhUUBgYjIiY1NDYzMhYXIw4CFRQWFjMyN
jU0JiMB7l1RcHcBDYFab3kuZk+VbHWFWmgLFO9nQzBoT15xcWMCXFqjtSMSTl2Ccj5oQbufs8JmWEguXEM8a0NyY250AAABAA
oAAAHOAsAABgAAAQEjASE1IQHO/ugVARz+TQHEAq79UgKuEgAAAwA3/+cCKQLHABcAIwAvAAASJjU0NjMyFhUUBgcWFhUUBiM
iJjU0NjckJiMiBhUUFjMyNjUEBhUUFjMyNjU0JiOVSnpra3pKT1Fcg3Z2g1xRAR1xYGBxcGFhcP7LgXxpaXyBZAF6Wz9RYmJR
P1sQDmFKXmxsXkphDvhTU05MVVVMs1ZaWGBgWFpWAAACADz/+AIgAscAGwApAAA2FjMyNjU0JwYGIyImNTQ2NjMyFhUUBiMiJ
iczPgI1NCYmIyIGFRQWM3hdUXB3AQ2BWm95LmZPlWx1hVpoCxTvZ0MwaE9ecXFjZFqjtSMSTl2Ccj5oQbufs8JmWEguXEM8a0
NyY250AP//ACD//gBUAfkAIgAEAAAABwAEAAABxQABABQAAAHvAhwABgAAJRMzAyMDMwEC2BXmD+YWHgH+/eQCHAABACgBcwG
UAYUAAwAAARUhNQGU/pQBhRISAAAAAAAAHAFoAAwABBAkAAACiAAAAAwABBAkAAQAYAKIAAwABBAkAAgAOALoAAwABBAkAAwA2
AMgAAwABBAkABAAYAKIAAwABBAkABQAKAP4AAwABBAkABgAYAQgAQwBvAHAAeQByAGkAZwBoAHQAIAAyADAAMgAwACAAVABoA
GUAIABQAG8AcABwAGkAbgBzACAAUAByAG8AagBlAGMAdAAgAEEAdQB0AGgAbwByAHMAIAAoAGgAdAB0AHAAcwA6AC8ALwBnAG
kAdABoAHUAYgAuAGMAbwBtAC8AaQB0AGYAbwB1AG4AZAByAHkALwBQAG8AcABwAGkAbgBzACkAUABvAHAAcABpAG4AcwAgAFQ
AaABpAG4AUgBlAGcAdQBsAGEAcgBJAFQARgBPADsAIABQAG8AcABwAGkAbgBzACAAVABoAGkAbgA7ACAANAAuADAAMAA0AGIA
OAA0AC4AMAAwADQAUABvAHAAcABpAG4AcwAtAFQAaABpAG4AAAADAAAAAAAA/7UAMgAAAAAAAAAAAAAAAAAAAAAAAAAAAAAEAA
AAKABwAHgABREZMVAAIAAQAAAAA//8AAAAAAAAAAQAAAAoALAAuAANERkxUABRkZXYyAB5kZXZhAB4ABAAAAAD//wAAAAAAAA
AAAAAAAAAABAAAAAA==) format('truetype');
  font-weight: 100;
}

/* ---------- 四套壁纸（=主题）10-04 定稿 ----------
   这四枚都是"整套色阶"：页面底、卡底、文字四级、chips、按钮、左侧那块方块
   全部收进同一支色相，所以方块色也在这里（镜像值），真身在 utils/palette.js 的 THEMES。
   两边逐字等值由色板那把尺子钉死：--bg-page=theme.page、--bg-card=theme.line、
   --btn-bg=chromeOf(key).sel。改一边必须改另一边，只改一边会在真机上出现
   "导航条是上一页那套色"这一类对不上。
   卡底不用纯白：白纸贴在米纸上像一张张贴纸，退到比页面底亮 9 档的那一支。
   色值出处：docs/design/配色统一-四套莫兰迪/生成色板 5.py（那一份是唯一算法，别在手底下改）。 */
.theme-tint-paper {
  --bg-page: #E2DED4;
  --bg-card: #F3F3F1;
  --bg-card-glass: rgba(243, 243, 241, 0.85);
  --text-primary: #352E1D;
  --text-secondary: rgba(53, 46, 29, 0.72);
  --text-tertiary: rgba(53, 46, 29, 0.72);
  --border: rgba(53, 46, 29, 0.09);
  --card-edge: rgba(53, 46, 29, 0.12);
  --accent: #352E1D;
  --accent-soft: rgba(53, 46, 29, 0.07);
  --btn-bg: #766948;
  --chip-idle: rgba(53, 46, 29, 0.07);
}

.theme-tint-celadon {
  --bg-page: #D4E2D7;
  --bg-card: #F1F3F2;
  --bg-card-glass: rgba(241, 243, 242, 0.85);
  --text-primary: #1D3522;
  --text-secondary: rgba(29, 53, 34, 0.72);
  --text-tertiary: rgba(29, 53, 34, 0.72);
  --border: rgba(29, 53, 34, 0.09);
  --card-edge: rgba(29, 53, 34, 0.12);
  --accent: #1D3522;
  --accent-soft: rgba(29, 53, 34, 0.07);
  --btn-bg: #467350;
  --chip-idle: rgba(29, 53, 34, 0.07);
}


.theme-tint-blush {
  --bg-page: #E2D4DB;
  --bg-card: #F3F1F2;
  --bg-card-glass: rgba(243, 241, 242, 0.85);
```

```
  --text-primary: #351D29;
  --text-secondary: rgba(53, 29, 41, 0.72);
  --text-tertiary: rgba(53, 29, 41, 0.72);
  --border: rgba(53, 29, 41, 0.09);
  --card-edge: rgba(53, 29, 41, 0.12);
  --accent: #351D29;
  --accent-soft: rgba(53, 29, 41, 0.07);
  --btn-bg: #915975;
  --chip-idle: rgba(53, 29, 41, 0.07);
}

/* 雨雾沿用旧 key gradient-blue / 类名 theme-blue：后端 WALLPAPER_PRESETS 那六个值一个都没动，
   旧值还收得下，所以压成四枚这件事零部署（别名表在 utils/palette.js 的 WALLPAPER_ALIAS）。
   这一枚 S 只有 13，是"冷灰"不是蓝——和站长 10-04 撤掉的那支雾蓝（S 21）不是一回事。 */
.theme-blue {
  --bg-page: #D7D9E0;
  --bg-card: #F1F2F3;
  --bg-card-glass: rgba(241, 242, 243, 0.85);
  --text-primary: #212431;
  --text-secondary: rgba(33, 36, 49, 0.72);
  --text-tertiary: rgba(33, 36, 49, 0.72);
  --border: rgba(33, 36, 49, 0.09);
  --card-edge: rgba(33, 36, 49, 0.12);
  --accent: #212431;
  --accent-soft: rgba(33, 36, 49, 0.07);
  --btn-bg: #616986;
  --chip-idle: rgba(33, 36, 49, 0.07);
}

/* 界面字体：不挂这三个类就是上面 page 那条苹方链（默认）。
   宋体把 STSong 放在 Songti SC 前面：Mac 模拟器上实测命中的是 STSong，'Songti SC' 点不中；
   iOS 真机没验过（站长那台是安卓），所以两个名字都留在链里，谁命中用谁。
   这一排只承诺 iOS——这几套字是苹果系统自带的。安卓机型一律回落系统黑体，
   站长 09-26 定的不为此内置字体包（"避免浪费开发资源"），界面文案直接写明仅 iPhone / iPad 可选。 */
.font-song { font-family: 'STSong', 'Songti SC', '宋体-简', 'Noto Serif CJK SC', 'Noto Serif SC',
'Source Han Serif SC', '思源宋体 CN', 'SimSun', 'Noto Serif', serif; }

.font-fang { font-family: 'STFangsong', 'FangSong', '仿宋', 'FangSong_GB2312', 'Noto Serif CJK SC',
'Source Han Serif SC', 'Noto Serif', serif; }

.font-kai { font-family: 'Kaiti SC', 'STKaiti', 'Kaiti', '楷体', 'KaiTi', 'TW-Kai', 'AR PL KaitiM
GB', 'Noto Serif CJK SC', 'Noto Serif', serif; }

.container {
  /* 顶部这 2rpx 不是间距，是堵外边距穿透的：子元素第一个的 margin-top 会和
     没有 padding-top 的父级合并，把整个容器往下推，露出 page 自己的那一底。
     浅色壁纸下看不出来，深色壁纸下导航条底下就是一条白带（深色那两枚 10-04 已撤）。 */
  padding: 2rpx var(--sp-3) 0;

  /* 主题类挂在 .container 上而不是 page 上（小程序给不了 page 类名），
     所以底色必须由容器自己画。真机实测：少了这两行，切壁纸只改导航条和 chip，整页底色永远不变。 */
  background: var(--bg-page);
  min-height: 100vh;
  box-sizing: border-box;
}

/* ---------- 卡：白底 + 描边，不投影 ---------- */
.card {
  position: relative;
  display: block;
  background: var(--bg-card);
  color: var(--text-primary);
  border: var(--w-edge) solid var(--card-edge);
  border-radius: var(--r-card);
```

```
  padding: var(--sp-3) var(--sp-4) var(--sp-4);
  margin-bottom: var(--sp-2);
  overflow: hidden;
}

.card:active {
  transform: scale(0.988);
}

/* ---------- 行卡：左侧方块 + 右侧正文。列表、详情头、入口列表共用这一套 ---------- */
.row {
  display: flex;
  align-items: flex-start;
  gap: var(--sp-row);
  padding: var(--sp-row);
}

.blk {
  flex: none;
  position: relative;
  width: var(--blk);
  height: var(--blk);
  box-sizing: border-box;
  padding: var(--blk-pad);
  border-radius: var(--r-block);
  background: var(--blk-bg, var(--accent));
  color: var(--blk-ink, #ffffff);
  border: var(--w-edge) solid var(--blk-edge, transparent);
  overflow: hidden;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}

/* 块内分类名：单行，顶到边就用渐隐收掉，不打省略号。

   渐隐只吃盒子最右 14%，而盒子宽度固定，所以短名字的末字落在渐隐区之前、完全不受影响，
   只有真被裁掉的长名字会淡出。这块只是提示：识别由颜色承担、阅读由右侧标题承担，
   所以裁掉尾巴不丢信息——中文 2~3 字永远到不了渐隐区，这条规则实际上只对英文生效。
   省略号会把一个被裁断的词变成半个词，渐隐是"这里还有，不必读"，所以选渐隐。 */
.blk-name {
  position: relative;
  z-index: 2;
  font-size: var(--fs-block, 32rpx);
  font-weight: 800;
  line-height: 1.2;
  letter-spacing: -0.6rpx;
  white-space: nowrap;
  overflow: hidden;
  -webkit-mask-image: linear-gradient(90deg, #000 0 86%, transparent 100%);
  mask-image: linear-gradient(90deg, #000 0 86%, transparent 100%);
}

.blk-date {
  position: relative;
  z-index: 2;
  font-size: var(--fs-micro);
  font-weight: 800;
  letter-spacing: 1.4rpx;
  opacity: 0.72;
}

.blk-body {
  flex: 1;
```

```
  min-width: 0;
}

.row-title {
  font-size: var(--fs-title);
  font-weight: 800;
  line-height: var(--lh-tight);
  letter-spacing: -0.5rpx;
  color: var(--text-primary);
  margin-bottom: 10rpx;
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 2;
  overflow: hidden;
}

.row-foot {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: var(--sp-2);
  font-size: var(--fs-tiny);

  font-weight: 700;
  color: var(--text-tertiary);
}

.row-foot .src {
  color: var(--text-secondary);
}

.row-foot .foot-l {
  min-width: 0;
  overflow: hidden;
  white-space: nowrap;
  text-overflow: ellipsis;
}

/* 方块的小尺寸变体：分类管理那一列色点。
   方块里的字用汉字而不是图标字体——项目没有 iconfont，单个汉字在苹方下必定渲染得出来，
   换成 ✎ ⌗ 这类符号就要赌设备字体。 */
.blk.tall {
  height: 200rpx;
}

.blk.dot {
  width: 52rpx;
  height: 52rpx;
  border-radius: 18rpx;
  padding: 0;
}

/* ---------- 抽象母题：由 palette.js 按笔记 id 选一个，位置大小也从 id 派生 ---------- */
.motif {
  position: absolute;
  right: calc(0px - var(--motif-shift));
  bottom: calc(0px - var(--motif-shift));
  width: var(--motif-size);
  height: var(--motif-size);
  pointer-events: none;
  /* 母题永远在文字下面。真机实测过：不给这一层，长标题的后半句会被条纹盖掉。 */
  z-index: 0;
}
```

```
.motif-circle {
  border-radius: 50%;
  background: var(--blk-motif);
}

.motif-arc {
  border-radius: 50%;
  border: 12rpx solid var(--blk-motif);
  border-right-color: transparent;

  border-bottom-color: transparent;
}

.motif-stripes {
  background: repeating-linear-gradient(
    115deg,
    var(--blk-motif) 0 12rpx,
    transparent 12rpx 26rpx
  );
}

.motif-dots {
  background-image: radial-gradient(var(--blk-motif) 6rpx, transparent 7rpx);
  background-size: 26rpx 26rpx;
}

.motif-pill {
  width: calc(var(--motif-size) * 1.5);
  height: calc(var(--motif-size) * 0.42);
  border-radius: var(--r-pill);
  background: var(--blk-motif);
}

/* ---------- chrome ---------- */
.chip {
  display: inline-block;
  padding: 14rpx 28rpx;
  border-radius: var(--r-pill);
  font-size: var(--fs-meta);
  font-weight: 600;
  background: var(--chip-idle);
  color: var(--text-secondary);
}

.chip.active {
  background: var(--tone-bg, var(--accent));
  color: var(--tone-ink, var(--bg-page));
}

.tag {
  display: inline-block;
  background: var(--chip-idle);
  color: var(--text-primary);
  font-size: var(--fs-tiny);
  font-weight: 700;
  padding: 6rpx 18rpx;
  border-radius: 14rpx;
  margin-right: var(--sp-1);
}


.btn-primary {
  background: var(--btn-bg);
  color: var(--btn-ink);
```

```
  border-radius: var(--r-pill);
  font-size: var(--fs-body);
  font-weight: 700;
  padding: var(--sp-3) 0;
  text-align: center;
}

.btn-secondary {
  background: var(--chip-idle);
  color: var(--text-primary);
  border-radius: var(--r-pill);
  font-size: var(--fs-body);
  font-weight: 600;
  padding: var(--sp-3) 0;
  text-align: center;
}

/* ---------- 卡内控件的面 ----------
   饱和色大卡里的输入框、按钮、多行区长什么样。--face / --face-ink / --solid-bg /
   --solid-ink 由 utils/palette.js 的 toneStyle 按这块色的深浅发下来，这里只写形状。
   深底卡（蓝）用半透明白面，亮底卡（橙/黄）用近白面 + 该卡墨色字。

   这份定义从 pages/create 提到全局，是因为首页的搜索卡要用同一套控件：
   两个页面各写一遍，早晚会走样成两种输入框，而"和新建那几张卡一致"恰恰是要求本身。 */
.face-box {
  display: flex;
  align-items: center;
  height: 88rpx;
  margin-top: 26rpx;
  padding: 0 var(--sp-3);
  border-radius: var(--r-btn);
  background: var(--face);
  color: var(--face-ink);
}

.face-input {
  flex: 1;
  min-width: 0;
  font-size: var(--fs-body);
  color: var(--face-ink);
}

.face-ph {
  color: var(--face-ph, var(--face-ink));
}

.face-x {

  flex: none;
  padding: 0 8rpx;
  font-size: var(--fs-meta);
  opacity: 0.6;
}

.face-area {
  width: 100%;
  height: 180rpx;
  margin-top: 16rpx;
  padding: 18rpx var(--sp-3);
  box-sizing: border-box;
  border-radius: var(--r-btn);
  background: var(--face);
  color: var(--face-ink);
  font-size: var(--fs-body);
```

```
  line-height: var(--lh-body);
}

.acts {
  display: flex;
  gap: var(--sp-2);
  margin-top: 20rpx;
}

.act {
  flex: 1;
  height: 88rpx;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: var(--r-pill);
  font-size: var(--fs-body);
  font-weight: 700;
}

.act.ghost { background: var(--face); color: var(--face-ink); }
.act.solid { background: var(--solid-bg); color: var(--solid-ink); }
/* 不可用：面变薄、字变淡，且和 solid 的差别一眼看得出来，不靠文字解释 */
.act.off { background: var(--face-off); color: var(--face-off-ink); }

/* ---------- 卡底功能图形 ----------
   几何取自 Lucide v1.50.0（ISC）：这里只放公共的"怎么把一张 SVG 变成压在水印位置的色"，
   具体形状（mask-image）由各页自己给。图形本体是一块 var(--blk-glyph) 的色，
   SVG 只当遮罩用，所以换分类色时图形跟着变，颜色不会在 WXSS 里出现第二份。 */
.glyph {
  position: absolute;
  right: -30rpx;
  bottom: -46rpx;
  width: 264rpx;

  height: 264rpx;
  pointer-events: none;
  z-index: 0;
  background: var(--blk-glyph);
  -webkit-mask-repeat: no-repeat;
  mask-repeat: no-repeat;
  -webkit-mask-position: center;
  mask-position: center;
  -webkit-mask-size: contain;
  mask-size: contain;
}

// ===== miniprogram/custom-tab-bar/index.js — 自定义标签栏逻辑 =====
const { chromeOf } = require('../utils/palette.js')

Component({
  data: {
    // 0 新建 / 1 笔记 / 2 我的。新建放最左是刻意的：新用户第一次进来落到的就是
    // 那三个色块，"往哪儿存"这件事不用先找入口。
    selected: 0,
    lang: 'zh',
    list: [],
    // 胶囊那一块面连同字色、发丝边、投影、选中那枚圆底：由 JS 递进来（组件读不到 page 上的
    // CSS 变量），值全部来自 palette.chromeOf——和首页那条搜索条是同一个函数，
    // 所以这两块面永远同色。以前这里是"墨黑常量 + 深色壁纸换一块写死的 #1a1c22"，
    // 四枚壁纸只有两种胶囊。
    chromeStyle: '',
  },
```

```
  attached() {
    this.updateLabels()
    const app = getApp()
    if (app.globalData.userInfo) {
      this.applyTheme(app.getWallpaper())
    }
  },

  methods: {
    applyTheme(wallpaper) {
      // 组件读不到 page 上的 CSS 变量，颜色只能由 JS 算好递进来；
      // 判定结果一律取 palette 里那份，不在这里另记一遍壁纸名单。
      this.setData({ chromeStyle: chromeOf(wallpaper).style })
    },

    updateLabels() {
      const { t } = require('../utils/i18n.js')
      const lang = getApp().globalData.userInfo?.language || 'zh'
      this.setData({
        lang,
        // text 不再上屏（三个标签 10-01 撤了），但它是 aria-label 和读屏的那一份，

        // 所以照旧跟着语言走，别处不读它。
        list: [
          {
            pagePath: '/pages/create/create',
            text: t('tabCreate', lang),
            icon: 'plus',
          },
          {
            pagePath: '/pages/index/index',
            text: t('tabNotes', lang),
            icon: 'rows3',
          },
          {
            pagePath: '/pages/me/me',
            text: t('tabMe', lang),
            icon: 'user',
          },
        ],
      })
    },

    switchTab(e) {
      const index = e.currentTarget.dataset.index
      const item = this.data.list[index]
      if (this.data.selected !== index) {
        wx.switchTab({ url: item.pagePath })
      }
    },
  },
})

<!-- ===== miniprogram/custom-tab-bar/index.wxml — 自定义标签栏模板 ===== -->
<view class="tab-bar" style="{{chromeStyle}}">
  <view
    wx:for="{{list}}"
    wx:key="index"
    class="tab-item {{selected === index ? 'active' : ''}}"
    data-index="{{index}}"
    bindtap="switchTab"
    aria-role="button"
    aria-label="{{item.text}}"
  >
```

```
    <!-- 站长 10-01：三个按钮去掉文字，选中态从"文字下面一条短线"改成"图标垫一枚圆底"。
         文字没有丢，它挪到 aria-label 上——读屏还问得出"这是哪一格"，屏幕上不再占一排。 -->
    <view class="tab-icon glyph-{{item.icon}}"></view>
  </view>
</view>

/* ===== miniprogram/custom-tab-bar/index.wxss — 自定义标签栏样式 ===== */
/* 底部导航：浮起的胶囊。

   这里是自定义组件，拿不到 page 上的 CSS 变量，所以颜色一律由 JS 递进来（style 上那串
   --chrome-*，值来自 palette.chromeOf）。
   界面字体那三个 .font-* 类在这里没有了：10-01 把三个标签的文字撤干净之后，
   这一整块组件里一个字都不排，字体栈自然用不上（app.wxss 那份是页面自己的，照旧）。 */
.tab-bar {
  display: flex;
  position: fixed;
  box-sizing: border-box;
  bottom: 20rpx;
  left: 24rpx;
  right: 24rpx;
  height: 108rpx;
  background: var(--chrome-bg);
  border: 3rpx solid var(--chrome-line);
  border-radius: 44rpx;
  padding-bottom: 0;
  /* 描边宽用 3rpx 而不是 var(--w-edge)：组件读不到 page 上的令牌，这个数在 app.wxss 里也是 3。
     深壁纸下这块面压页面底不到 3，边界靠的就是这一圈纸白 @22%，不是靠投影。 */
  box-shadow: 0 16rpx 44rpx var(--chrome-shadow);
  z-index: 999;
}

.tab-item {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  /* 选中/未选中只换 currentColor 这一件事：图标那三条描边、那根横杠、那个加号全部吃它。
     圆底那一块面吃 --chrome-sel（同一支色只抬明度，见 palette.chromeOf）。 */
  color: var(--chrome-idle);
}

.tab-item.active {
  color: var(--chrome-ink);
}

/* 图标本身 40rpx（= 真机约 21px，描边 1.6/24 落下来正好 1.4px，不发虚也不糊），
   垫底那一枚圆 76rpx：108 高的胶囊上下各留 16，圆不会顶到边。 */
.tab-icon {
  width: 76rpx;
  height: 76rpx;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  background: transparent;
  transition: background-color 0.18s ease;
}

.tab-item.active .tab-icon {

  background: var(--chrome-sel);
}

.tab-icon::before {
```

```
  content: '';
  width: 40rpx;
  height: 40rpx;
  background-color: currentColor;
  -webkit-mask-repeat: no-repeat;
  mask-repeat: no-repeat;
  -webkit-mask-position: center;
  mask-position: center;
  -webkit-mask-size: contain;
  mask-size: contain;
}

/* ---------- 三枚图形 ----------
   几何取自 Lucide v1.49.0（ISC 授权，可商用）：plus / rows-3 / user-round。
   和新建页那几枚是同一套：24 格、同一描边重量、圆头线帽。
   做法沿用现网那一条——形状走 mask，颜色由 currentColor 给，所以换壁纸不用换图。
   为什么不是 PNG：PNG 得把颜色钉死在像素里，而底栏那三枚的色要跟着四套壁纸走
   （未选中纸白 @62%、选中纸白 @98%）；同一支形状一份 mask 就够，两份 PNG 只是翻倍。
   站长要的是"国外图标站那套精致图形"，图形就是那套，承载方式换成能跟着主题换色的这种。 */
.glyph-plus::before {
  -webkit-mask-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 24 24' fill='none' stroke='%23000' stroke-width='1.6' stroke-linecap='round' stroke-
linejoin='round'%3E%3Cpath d='M5 12h14' /%3E %3Cpath d='M12 5v14' /%3E%3C/svg%3E");
  mask-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24
24' fill='none' stroke='%23000' stroke-width='1.6' stroke-linecap='round' stroke-
linejoin='round'%3E%3Cpath d='M5 12h14' /%3E %3Cpath d='M12 5v14' /%3E%3C/svg%3E");
}

.glyph-rows3::before {
  -webkit-mask-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 24 24' fill='none' stroke='%23000' stroke-width='1.6' stroke-linecap='round' stroke-
linejoin='round'%3E%3Crect width='18' height='18' x='3' y='3' rx='2' /%3E %3Cpath d='M21 9H3'
/%3E %3Cpath d='M21 15H3' /%3E%3C/svg%3E");
  mask-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24
24' fill='none' stroke='%23000' stroke-width='1.6' stroke-linecap='round' stroke-
linejoin='round'%3E%3Crect width='18' height='18' x='3' y='3' rx='2' /%3E %3Cpath d='M21 9H3'
/%3E %3Cpath d='M21 15H3' /%3E%3C/svg%3E");
}

.glyph-user::before {
  -webkit-mask-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
viewBox='0 0 24 24' fill='none' stroke='%23000' stroke-width='1.6' stroke-linecap='round' stroke-
linejoin='round'%3E%3Ccircle cx='12' cy='8' r='5' /%3E %3Cpath d='M20 21a8 8 0 0 0-16 0'
/%3E%3C/svg%3E");
  mask-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 24
24' fill='none' stroke='%23000' stroke-width='1.6' stroke-linecap='round' stroke-
linejoin='round'%3E%3Ccircle cx='12' cy='8' r='5' /%3E %3Cpath d='M20 21a8 8 0 0 0-16 0'
/%3E%3C/svg%3E");
}

// ===== miniprogram/utils/api.js — 网络请求封装 =====
// 接口地址属于前后端之间的那道边界，只在这里出现一次。
// 之前它挂在 app.globalData.apiBase 上，页面拼 URL 时还得绕回 getApp() 去取。
const API_BASE = 'https://api.agentsbin.cn/wtsj'

// 私密笔记的解锁凭证。服务端在"验对密码"那一步发一条 15 分钟短命的 token，
// 之后每个请求带上它，服务端才把私密笔记的正文和概要发下来——以前这道锁只在界面里，
// 同一个登录令牌直接打 GET /api/notes/{id} 就能拿到整篇正文。
// 只存内存（挂在 globalData 上，不落 storage）：杀了小程序重进就要重新输，
// 这一闸防的恰恰是"手机在别人手里"那一段时间。
// 本机判的有效期故意比服务端那 15 分钟短一分钟：卡在边界上时宁可多问一次密码，
// 也不要出现"客户端以为还解锁着、服务端已判未解锁"，那会让详情页拿到一份裁过的空壳。
```

```
const UNLOCK_TTL_MS = 14 * 60 * 1000

function _unlockToken() {
  const app = getApp()
  const u = app.globalData.privateUnlock
  if (!u || !u.token) return ''
  if (Date.now() > u.expireAt) {
    app.globalData.privateUnlock = null
    return ''
  }
  return u.token
}

function setPrivateUnlock(token) {
  const app = getApp()
  app.globalData.privateUnlock = token ? { token, expireAt: Date.now() + UNLOCK_TTL_MS } : null
}

function clearPrivateUnlock() {
  getApp().globalData.privateUnlock = null
}

function hasPrivateUnlock() {
  return !!_unlockToken()
}

function _headers(contentType, extra) {
  const h = {
    'content-type': contentType || 'application/json',
  }
  const app = getApp()
  if (app.globalData.token) {
    h['Authorization'] = `Bearer ${app.globalData.token}`
  }
  const unlock = _unlockToken()
  if (unlock) h['X-Private-Token'] = unlock
  if (extra) Object.assign(h, extra)
  return h
}

const request = (url, method, data, options = {}, retryCount = 0) => {
  return new Promise((resolve, reject) => {
    wx.request({
      url: `${API_BASE}${url}`,
      method: method || 'GET',
      data: data || {},
      header: _headers(options.contentType, options.header),
      timeout: 30000,
      success(res) {
        if (res.statusCode >= 200 && res.statusCode < 300) {

          resolve(res.data)
        } else if (res.statusCode === 401) {
          // 注销这一类不可逆的写操作不带静默重登：401 之后换一个新账号重试，删掉的就是
          // 那个新号，用户看到"已注销"而自己的旧数据还在原地。宁可让他重进一次小程序。
          if (options.noRelogin) {
            reject(res)
            return
          }
          // Prevent infinite relogin loop: only retry once
          if (retryCount >= 1) {
            reject(new Error('认证失败，请重新登录'))
            return
          }
```

```
          // Clear auth state on 401 and trigger re-login
          const app = getApp()
          app.globalData.token = ''
          app.globalData.userId = ''
          app.globalData.userInfo = null
          app.globalData.isLoggedIn = false
          app.globalData.loginPromise = null // Reset login promise to allow retry
          // 解锁凭证跟着登录态一起清：重登之后可能是另一个 generation 的账号，
          // 留着旧凭证只会让下一次读私密笔记拿到一份裁过的空壳。
          app.globalData.privateUnlock = null

          // Trigger login and retry after successful authentication
          app.getLoginPromise()
            .then(() => {
              // Retry original request with new token (increment retry count)
              return request(url, method, data, options, retryCount + 1)
            })
            .then(resolve)
            .catch(reject)
        } else {
          reject(res)
        }
      },
      fail(err) {
        reject(err)
      }
    })
  })
}

const uploadSingleImage = (url, filePath, formData, retryCount = 0) => {
  return new Promise((resolve, reject) => {
    wx.uploadFile({
      url: `${API_BASE}${url}`,
      filePath,
      name: 'images',

      header: _headers(),
      formData,
      success(res) {
        if (res.statusCode >= 200 && res.statusCode < 300) {
          resolve(JSON.parse(res.data))
        } else if (res.statusCode === 401) {
          // Prevent infinite relogin loop: only retry once
          if (retryCount >= 1) {
            reject(new Error('认证失败，请重新登录'))
            return
          }

          // Handle 401 for uploads: clear auth and trigger re-login
          const app = getApp()
          app.globalData.token = ''
          app.globalData.userId = ''
          app.globalData.userInfo = null
          app.globalData.isLoggedIn = false
          app.globalData.loginPromise = null

          app.getLoginPromise()
            .then(() => {
              // Retry upload with new token (increment retry count)
              return uploadSingleImage(url, filePath, formData, retryCount + 1)
            })
            .then(resolve)
```

```
            .catch(reject)
        } else {
          // wx.uploadFile 拿到的 data 是字符串，而 wx.request 的是对象。不统一成对象，
          // 页面里 err.data.detail 这种取法在上传失败时会拿到 undefined，用户只能看到兜底文案。
          try {
            reject(Object.assign({}, res, { data: JSON.parse(res.data) }))
          } catch (e) {
            reject(res)
          }
        }
      },
      fail(err) {
        reject(err)
      }
    })
  })
}

const ingestScreenshots = async (filePaths) => {
  let batchId = null
  for (const filePath of filePaths) {
    const formData = batchId ? { batch_id: batchId } : undefined
    const res = await uploadSingleImage('/api/ingest/screenshots/stage', filePath, formData)
    batchId = res.batch_id

  }
  return request('/api/ingest/screenshots/process', 'POST', { batch_id: batchId }, { contentType:
'application/x-www-form-urlencoded' })
}

const getTaskStatus = (taskId) => request(`/api/tasks/${taskId}`)

const pollTask = (taskId, intervalMs = 2000, maxWaitMs = 300000) => {
  return new Promise((resolve, reject) => {
    const start = Date.now()
    const poll = () => {
      if (Date.now() - start > maxWaitMs) {
        reject({ timeout: true })
        return
      }
      getTaskStatus(taskId)
        .then((data) => {
          if (data.status === 'completed') {
            resolve(data.result)
          } else if (data.status === 'failed') {
            reject(data.result || { error: 'unknown' })
          } else {
            setTimeout(poll, intervalMs)
          }
        })
        .catch(reject)
    }
    poll()
  })
}

module.exports = {
  request,
  getNotes: (skip = 0, limit = 20, categoryId, search) => {
    let url = `/api/notes/?skip=${skip}&limit=${limit}`
    if (categoryId != null) url += `&category_id=${categoryId}`
    if (search) url += `&search=${encodeURIComponent(search)}`
    return request(url)
  },
```

```
  getNote: (id) => request(`/api/notes/${id}`),
  createNote: (data) => request('/api/notes/', 'POST', data),
  updateNote: (id, data) => request(`/api/notes/${id}`, 'PUT', data),
  deleteNote: (id) => request(`/api/notes/${id}`, 'DELETE'),
  ingestUrl: (url) => request('/api/ingest/url', 'POST', { url }, { contentType: 'application/x-
www-form-urlencoded' }),
  // 手打这一档也交给模型提炼：送的是用户自己写的标题 + 原文，回来的是摘要。
  // 归类跟着这一篇走（服务端认人不认客户端说的 source_type）。
  ingestText: (data) => request('/api/ingest/text', 'POST', data),
  ingestScreenshots,
  getTaskStatus,
  pollTask,
  getCategories: () => request('/api/categories/'),

  createCategory: (data) => request('/api/categories/', 'POST', data),
  updateCategory: (id, data) => request(`/api/categories/${id}`, 'PUT', data),
  deleteCategory: (id) => request(`/api/categories/${id}`, 'DELETE'),
  reorderCategories: (ids) => request('/api/categories/reorder', 'POST', { ids }),
  // 昵称是"分享形象"里用户自己填的，只在主动分享这一刻才跟着上服务器，
  // 和标题摘要同性质——都是他选择公开的内容，不是我们采集的账号资料。
  createShare: (noteId, authorName) =>
    request('/api/shares/', 'POST', { note_id: noteId, author_name: authorName || null }),
  // 把别人分享页上那一条整份抄进自己库。来源信息由服务端钉在笔记上，客户端碰不到它。
  importFromShare: (token) => request('/api/notes/from-share', 'POST', { token }),
  // 分享状态：详情页拿它决定要不要显示"撤掉分享"这一行
  getShareStatus: (noteId) => request(`/api/shares/status?note_id=${noteId}`),
  // 撤掉之后那张码扫开就是"分享已关闭"，再点分享会给一张新码
  revokeShare: (noteId) => request('/api/shares/revoke', 'POST', { note_id: noteId }),
  // 分享落地页原来直接调 api.request 拼路径，等于绕过了这一层
  getShare: (token) => request(`/api/shares/${encodeURIComponent(token)}`),
  getShareQRCodeUrl: (token) => `${API_BASE}/api/shares/${token}/qrcode`,
  getWallpaperOptions: () => request('/api/user/wallpaper/options'),
  // 卡片模板配方下发（P0-5）。路径尾巴那个斜杠是必须的：FastAPI 那里注册的是 ""，
  // 少一个斜杠服务端回 307，而 wx.request 对重定向的处理在各端不一致——别拿它赌。
  // 现网那张表还没建（这一批没部署），所以现在打过来只会 404，
  // posterTemplates.refresh 那边把任何失败都当"这次没拿到"，包内那十套照样画。
  getPosterTemplates: () => request('/api/poster/templates/'),
  getQuota: () => request('/api/user/quota'),
  // 确认标志必须在服务端看得见的地方，所以是 POST 带 body，不是 DELETE 带 body
  deactivateAccount: () =>
    request('/api/user/deactivate', 'POST', { confirm: true }, { noRelogin: true }),
  updateWallpaper: (wallpaper) => request('/api/user/wallpaper', 'PUT', { wallpaper }),
  updateLanguage: (language) => request('/api/user/language', 'PUT', { language }),
  setPrivatePassword: async (password) => {
    // 换密码会让服务端手上那批解锁凭证当场作废（凭证绑在旧密码摘要上），
    // 本机这份也跟着清掉，别留一条"界面以为还解锁着、服务端已经不认"的凭证。
    const r = await request('/api/user/private-password', 'PUT', { password })
    clearPrivateUnlock()
    return r
  },
  // 验对密码 = 拿到解锁凭证。收口在这里而不是让各页面自己记得存，
  // 是因为漏存的那一处症状很隐蔽：详情页能打开，但正文和概要是空的。
  verifyPrivatePassword: async (password) => {
    const r = await request('/api/user/private-password/verify', 'POST', { password })
    if (r && r.unlock_token) setPrivateUnlock(r.unlock_token)
    return r
  },
  getPrivatePasswordStatus: () => request('/api/user/private-password'),
  // 重置走 POST 不走 DELETE：与注销那条同一条理由——动作要在服务端日志里看得见实体。
  resetPrivatePassword: async () => {
    const r = await request('/api/user/private-password/reset', 'POST', {})
    clearPrivateUnlock()
    return r
```

```
  },

  // 解锁凭证这一头一尾三个口：详情页验完密码要重取列表时用它判"现在解没解锁"，
  // 退出登录那一步要清掉它，别把上一账号的解锁状态留给下一个。
  setPrivateUnlock,
  clearPrivateUnlock,
  hasPrivateUnlock,
}

// ===== miniprogram/utils/palette.js — 配色系统 =====
// 色块方向的色板与封面生成规则（当前是 V2 左块右文）。
//
// 三条硬规则从 V1 原样继承，都是实测或推理出来的，别随手改：
// ① 六个高饱和色各自固定配一种字色。黄/橙/绿配白字实测只有 1.63 / 3.03 / 2.98，
//    直接看不清，所以这三块配深墨字；蓝/紫/墨配白字。配对关系写死在下面。
// ② 颜色由分类决定，同分类恒定同色；未分类一律墨黑，让"没归类"这件事一眼可见。
// ③ 卡里的抽象母题（圆/弧/条带/点阵/药丸）由笔记 id 取模决定，位置大小也从 id 派生。
//    所以同一条笔记永远画成同一个样子，同分类色相一致但构图不重样。
//    全部用 CSS 画，不生成文件、不调 AI 生图。
//
// V1→V2 变的只有承载方式：颜色从整张卡退到卡片左侧那块方块，正文回白底。
// 所以母题现在住在方块里（方块自己 overflow:hidden），尺寸区间按方块重算。

const TONES = [
  { name: '芥末黄', bg: '#F6C445', ink: '#2A2005' },
  { name: '宝蓝', bg: '#3F52D6', ink: '#FFFFFF' },
  { name: '橙', bg: '#E9723D', ink: '#2C1204' },
  { name: '草绿', bg: '#46A863', ink: '#06260F' },
  { name: '紫', bg: '#6E4BD0', ink: '#FFFFFF' },
]
const UNCATEGORIZED = { name: '墨黑', bg: '#23252C', ink: '#FFFFFF' }

// Tips 行前面那枚"小灯泡"用的黄。值与上面芥末黄同一支，但**不写成 TONES[0]**：
// 那五支是"分类身份"，谁排第一随时会因排序调整而变，提示点不该跟着它走。
// 也不许抄进 WXSS——`验-统一录入条.js` 有一条专门扫 wxss 里有没有色板里的饱和色。
const TIP_DOT = '#F6C445'

/**
 * 纸片墙那一族莫兰迪（站长 10-02 指定"经典色卡里那四枚"）。
 * 为什么单开一族而不吃 TONES 那五支：一枚 150rpx 见方的纸片整块铺饱和色会把标题抢掉，
 * 而 TONES 是"分类身份"那一套小面（点、chip、模板圆点）共用的，改它等于改三个地方。
 * 四组字/底对比实测 5.21 / 6.72 / 6.68 / 7.54，都在正文档那道 4.5 以上。
 */
const PAPERS = [
  { name: '炭灰蓝', bg: '#5B6470', ink: '#EDEFF2' },
  { name: '雾蓝', bg: '#A8BCC9', ink: '#23323C' },
  { name: '灰绿', bg: '#A9BCA6', ink: '#22331F' },
  { name: '藕荷', bg: '#D3B6B4', ink: '#3B2523' },
]
/**
 * 取哪一枚：吃**序号**而不是 categoryId。序号≤0（或未给/非数）=未分类，钉死第一枚炭灰蓝；
 * 分类按它们在分类表里的顺序往后排（第一个分类→雾蓝、第二个→灰绿、第三个→藕荷），

 * 排到第四枚起在这三枚里循环——**不回到炭灰蓝**，那一枚是"没归类"专用身份，
 * 让一个真分类跟它撞色，一屏上就分不清哪篇没归类了。
 * 为什么不拿 id 取模：站长那四枚是按"未分类/旅游/生活/私密"这个顺序点的色，
 * 而 id 是数据库自增的，取模会把 旅游 和 生活 撞成同一枚——同一屏两枚同色就不是分类身份了。
 * 与 toneColor(i) 是同一条做法：序号进、色出，界面上不写第二份色值。
 */
const paperSkinFor = (ordinal) => {
  const n = Number(ordinal)
  const cyc = PAPERS.length - 1
  const p = ordinal == null || isNaN(n) || n <= 0
    ? PAPERS[0]
```

```
    : PAPERS[1 + ((n - 1) % cyc)]
  return { bg: p.bg, ink: p.ink, name: p.name }
}
/**
 * 「已分享」那一枚色块（站长 10-02：一行模式右侧只留这一种状态，其他状态什么都不画）。
 * 底色吃 TONES[1] 那支宝蓝——效果图上那架纸飞机用的就是它，飞机撤了、颜色留下。
 * 白字压上去实测 6.2:1。和 TIP_DOT 同一条规矩：只从 palette 出，不许抄进 WXSS。
 */
const SHARED_TAG = { bg: TONES[1].bg, ink: '#FFFFFF' }

/**
 * 背景形象图的深浅三档（站长 10-02 定：点一下换一档，走到最沉那档再回最亮；
 * 界面上就是一枚灰度实心点，不做形状——月相那版点他看过否了）。
 *
 * veil = 现网那层压暗罩（`.page-scrim` / `.head-scrim`）铺几层，直接拧它的 opacity。
 *        0 就是**一点不压**，照片等于原图（他拿原图对过：我上一版"纯白"档实测只有原图亮度的
 *        62%，因为那层 0.42~0.72 的罩子还挂着——那一版把"不叠新层"当成了"不压暗"，是两回事）。
 *        1 就是这半年一直在跑的那七个停点，一个字没改。
 * dot  = 界面上那一枚点的灰度：点本身就是当前档，越沉的档点越黑。
 *
 * 默认停在中档（DIM_DEFAULT，站长 10-02 夜里改的口径）：他原话"三档默认中档就行了，
 * 把调亮度选择权给用户"。所以这一档既不压到最沉（现网那半年的样子），也不放到原图，
 * 取中间那一格；本机键被别的版本写成脏值也兜回这一档。
 * 这一组数与 TIP_DOT 同一条规矩：只从 palette 出，不许抄进 WXSS。
 */
const BG_DIMS = [
  { v: 0, name: '满月', veil: 0, dot: '#FFFFFF' },
  { v: 1, name: '半月', veil: 0.5, dot: '#A7ABB2' },
  { v: 2, name: '弯月', veil: 1, dot: '#5C6169' },
]
const DIM_DEFAULT = BG_DIMS[1]

const dimAt = (v) => BG_DIMS.find((d) => d.v === v) || DIM_DEFAULT
// 点一下换到下一档：满月 → 半月 → 弯月 → 满月。
const dimNext = (v) => BG_DIMS[(BG_DIMS.indexOf(dimAt(v)) + 1) % BG_DIMS.length].v
// 弯月那一档不发样式：CSS 里那层罩子本来就是全铺的，写个 opacity:1 是废话。
const dimScrimStyle = (v) => (dimAt(v).veil === 1 ? '' : `opacity:${dimAt(v).veil}`)
const dimDotStyle = (v) => `background:${dimAt(v).dot}`


/**
 * 壁纸名。中文是两字釉色本名（壁纸条只有"色块 + 名字"那么大地方），
 * 英文侧站长 10-01 晚补的：这一排以前在英文态还是八个汉字。
 * 一律收成单个词——那一格不放得下两行。
 */
function themeLabel(key, lang) {
  const theme = themeOf(key)
  if (!theme) return ''
  return lang === 'en' ? theme.labelEn || theme.label : theme.label
}

const MOTIFS = ['motif-circle', 'motif-arc', 'motif-stripes', 'motif-dots', 'motif-pill']

function hexToRgb(hex) {
  const h = (hex || '').replace('#', '')
  const s = h.length === 3 ? h.split('').map((c) => c + c).join('') : h
  const n = parseInt(s.slice(0, 6), 16)
  return [(n >> 16) & 255, (n >> 8) & 255, n & 255]
}

function toHex(rgb) {
  return '#' + rgb.map((v) => Math.max(0, Math.min(255, Math.round(v))).toString(16).padStart(2,
'0')).join('')
}
```

```
// weightOfA = 第一个颜色占多少。想让某个颜色"只渗进去一点"，给它一个小权重。
function mix(a, b, weightOfA) {
  const ca = hexToRgb(a)
  const cb = hexToRgb(b)
  return toHex(ca.map((v, i) => v * weightOfA + cb[i] * (1 - weightOfA)))
}

// 给一个 hex 加透明度。海报上"同一张图洗一层某色"（波普套色、人像上的色罩）靠它，
// 免得每个模板自己手写 rgba 字面量，色板就又散了。
function withAlpha(hex, a) {
  const [r, g, b] = hexToRgb(hex)
  return `rgba(${r},${g},${b},${a})`
}

function hash(n) {
  // 把 id 打散成一个大整数，避免相邻 id 拿到相邻构图。
  const x = (Math.abs(Number(n) || 0) + 1) * 2654435761
  return x % 2147483647
}

/**
 * 分类对应的那组色。blockSkinFor 给的是 CSS 变量串，画布要的是裸色值，所以单独露一个。
 */
function toneFor(categoryId) {
  return categoryId == null ? UNCATEGORIZED : TONES[Math.abs(Number(categoryId)) % TONES.length]
}


/**
 * 生成一条笔记左侧那块方块的外观。
 * @param categoryId 笔记所属分类；null 走墨黑
 * @param noteId 笔记 id，决定构图
 * @returns {{style: string, motif: string}} style 直接塞进方块的 style 属性，motif 是母题类名
 */
function blockSkinFor(categoryId, noteId) {
  const tone = toneFor(categoryId)
  const ramp = rampFor(categoryId)
  const bg = ramp ? ramp.bg : tone.bg
  const ink = ramp ? ramp.ink : tone.ink
  const h = hash(noteId)
  const motif = MOTIFS[h % MOTIFS.length]
  // 母题只比底色暗/亮一点点，做质感不做主角
  const motifColor = mix(ink, bg, 0.16)
  // 墨黑这块在深色壁纸下会和卡片底糊在一起，只有它有描边；亮底上这条描边看不出来，所以两端都安全。
  // 色阶主题反过来：卡底是亮纸，最浅那两档（与卡底只差 1.2~1.6 的对比）会直接在卡上化掉，
  // 所以这里给一条"取 bg 和 ink 里更深那个"的细描边——浅档勾深边、深档勾深底边，都不刺眼。
  const edge = ramp
    ? withAlpha(lumOf(bg) < lumOf(ink) ? bg : ink, 0.22)
    : tone === UNCATEGORIZED ? 'rgba(255,255,255,0.16)' : 'transparent'
  const size = 108 + (h % 4) * 16 // rpx：108~156，方块 176rpx 见 app.wxss --blk
  const shift = 24 + (h % 3) * 8 // rpx：出界多少，从 id 派生
  const style = [
    `--blk-bg:${bg}`,
    `--blk-ink:${ink}`,
    `--blk-motif:${motifColor}`,
    `--blk-edge:${edge}`,
    `--motif-size:${size}rpx`,
    `--motif-shift:${shift}rpx`,
  ].join(';')
  return { style, motif }
}

/**
```

```
 * 只取某一组色的 CSS 变量串。给非笔记类的色块用（比如新建页那三张大卡、
 * 分类管理那列色点），免得有人在 WXSS 里另抄一份十六进制。
 * 连 --blk-motif 一起给：淡一层同色系图形要用的就是它，值仍从这一处派生。
 */
function toneStyle(i) {
  const tone = TONES[Math.abs(Number(i) || 0) % TONES.length]
  const ramp = rampFor(i)
  const bg = ramp ? ramp.bg : tone.bg
  const ink = ramp ? ramp.ink : tone.ink
  // --blk-glyph 比 --blk-motif 重一档（0.22 对 0.16）：新建页那三个图形要认得出来，
  // 但仍是压在卡底的水印，不是贴在表面的贴纸。
  //
  // 后面这组是"卡内控件的面"。新建页把输入框和按钮装进了饱和色大卡，
  // 面上用什么色取决于这块色本身是深还是浅：深底卡（字是白的）用半透明白面，

  // 亮底卡（字是深墨）用近白面 + 该卡自己的墨色字，否则黄/橙上放白半透明面会糊成一片。
  const onDark = inkIsLighter(bg, ink)
  return [
    `--blk-bg:${bg}`,
    `--blk-ink:${ink}`,
    `--blk-motif:${mix(ink, bg, 0.16)}`,
    `--blk-glyph:${mix(ink, bg, 0.22)}`,
    `--face:${onDark ? 'rgba(255,255,255,0.16)' : 'rgba(255,255,255,0.78)'}`,
    `--face-ink:${onDark ? '#FFFFFF' : ink}`,
    `--face-off:${onDark ? 'rgba(255,255,255,0.24)' : 'rgba(255,255,255,0.34)'}`,
    `--face-off-ink:${onDark ? 'rgba(255,255,255,0.5)' : mix(ink, bg, 0.45)}`,
    `--solid-bg:${onDark ? '#FFFFFF' : ink}`,
    `--solid-ink:${onDark ? bg : '#FFFFFF'}`,
    // 校验/权限提示那行字直接坐在卡色上，所以亮底卡用深红、深底卡用浅红，两边都够对比度。
    // 深红这一支原来也是手挑的（色阶那套 #7A1A13）：甲档第三档比原来暗了 3 档，同一个红掉到
    // 3.44~3.54（要 4.5），所以改成现算——enforce 只往"更深"的方向挪 L，挪到刚好过线为止，
    // 浅一档的卡色上它仍然是原来那个红，不会被无谓地压深。
    `--blk-err:${onDark ? '#FFD7D3' : enforce(ramp ? '#7A1A13' : '#8C2119', bg, 4.5)}`,
  ].join(';')
}

/**
 * 某一档的裸底色。
 * @param i 档号（和分类号同一套取模规则）
 * @param themeKey 画"别的主题"时必须传：壁纸选择器那一排缩略图每格画的都是另一套主题，
 *                 不传就会拿当前激活的主题去染色——停在象牙时去看天青那一格，
 *                 不带 ramp 的那几枚壁纸的缩略图也会跟着变成色阶色（真机截图里抓到过）。
 */
function toneColor(i, themeKey) {
  const theme = themeKey ? themeOf(themeKey) : ACTIVE_THEME
  const ramp = theme && theme.ramp
  if (ramp) return ramp.steps[Math.abs(Number(i) || 0) % ramp.steps.length]
  return TONES[Math.abs(Number(i) || 0) % TONES.length].bg
}

/**
 * 只给一对裸色，不给 --blk-bg。
 * 中性面板里想借用分类色（比如详情页的序号圆点）时用这个：
 * 直接贴方块那串会把整块中性面板染成色块。
 */
function toneVars(categoryId) {
  const tone = toneFor(categoryId)
  const ramp = rampFor(categoryId)
  return `--tone-bg:${ramp ? ramp.bg : tone.bg};--tone-ink:${ramp ? ramp.ink : tone.ink}`
}

/**
 * 四套壁纸（=主题），10-04 定稿。page/line 必须和 app.wxss 里 .theme-* 的
 * --bg-page/--bg-card 逐字等值，--btn-bg 同理必须等于 chromeOf(key).sel
```

```
 * （app.wxss 是 CSS、引不了 JS，镜像值是既有做法，等值由色板那把尺子逐枚钉死）。

 * 之所以在 JS 里再存一份：导航条颜色只能由 wx.setNavigationBarColor 传值，
 * 壁纸选择器也要靠它画缩略图，两处都不能猜 CSS 变量。
 *
 * 四枚都是"整套色阶"主题（都带 ramp）：不只换页面底，连左侧方块、新建页那三张大卡、
 * 按钮和标签都收进同一支色相，五档明度对应原来那五支彩色。
 * 色相种子不是我挑的：象牙/天青沿用现网那两枚的 H，樱落＝站长给的莫兰迪参考卡
 * 「樱落」那一格（R212 G172 B191），雨雾＝同一张卡的「豪雨」（R189 G192 B201）。
 * 取数规则三条，全部写在 docs/design/配色统一-四套莫兰迪/生成色板 5.py，那一份是唯一出处；
 * 改任何一条都要回生成器重算，别在手底下改 hex：
 * ① 页面底 L86（站长在甲 L86 / 乙 L78 里挑了甲），卡底比页面底亮 9 档。
 * ② 浅三档**锚卡底**、离卡底 11.8 / 21.4 / 33.9 个 L—照的是现网那套他认过的象牙 ramp
 *    的实测距离。锚页面底会出事：甲档下第一档离卡底只剩 2.7 档、对比 1.06，
 *    那块 176rpx 的色块直接化在卡上（v3 就踩了这个，10-04 第二轮量出来才改）。
 * ③ 深两档按 WCAG 反解（方块上那行分类名 16px/800 算小字，门槛 4.5），第 5 档再压 6 档。
 *
 * 四枚一律走服务端持久化（站长 10-04 拍「统一」）：后端 WALLPAPER_PRESETS 已把这四个 key
 * 都收进白名单，所以这里不再有 local 标记、app.js 里那套"只存本机"的旁路也撤干净了。
 * 雨雾沿用 gradient-blue 那个旧 key，象牙/天青/樱落是新加的三枚—旧六个值一个都没删，
 * 存量账号下次 PUT 不会被 400。（方案文档 §5.4 那条不一致到此结案。）
 *
 * line / lineEdge 只给缩略图里那两条中性描边行用：缩略图每一格画的都是"别的主题"，
 * 它拿不到当前主题的 CSS 变量，六个值必须由 JS 一个个带进去。
 */
const THEMES = [
  {
    key: 'tint-paper', cls: 'theme-tint-paper', label: '象牙', labelEn: 'Ivory',
    page: '#E2DED4', line: '#F3F3F1', lineEdge: 'rgba(53, 46, 29, 0.13)', dark: false,
    ramp: {
      steps: ['#DCD8CC', '#C9C1AE', '#AFA488', '#766B50', '#645B43'],
      inks: ['#352E1D', '#352E1D', '#352E1D', '#F2EFE9', '#F2EFE9'],
      uncategorized: { bg: '#352E1D', ink: '#F2EFE9' },
    },
  },
  {
    key: 'tint-celadon', cls: 'theme-tint-celadon', label: '天青', labelEn: 'Celadon',
    page: '#D4E2D7', line: '#F1F3F2', lineEdge: 'rgba(29, 53, 34, 0.13)', dark: false,
    ramp: {
      steps: ['#CCDCCF', '#AEC9B4', '#88AF91', '#4F7557', '#43634A'],
      inks: ['#1D3522', '#1D3522', '#1D3522', '#F2EFE9', '#F2EFE9'],
      uncategorized: { bg: '#1D3522', ink: '#F2EFE9' },
    },
  },
  {
    key: 'tint-blush', cls: 'theme-tint-blush', label: '樱落', labelEn: 'Blush',
    page: '#E2D4DB', line: '#F3F1F2', lineEdge: 'rgba(53, 29, 41, 0.13)', dark: false,
    ramp: {
      steps: ['#DCCCD4', '#C9AEBB', '#AF889C', '#8C5F76', '#7A5266'],
      inks: ['#351D29', '#351D29', '#351D29', '#F2EFE9', '#F2EFE9'],
      uncategorized: { bg: '#351D29', ink: '#F2EFE9' },
    },
```

（以上为前 30 页，以下为后 30 页）

```
        if text.strip():
            pages.append(text)
    if len(pages) < 2:
        return pages[0] if pages else ""
    return "\n\n".join(f"--- 第{i + 1}页 ---\n{text}" for i, text in enumerate(pages))

_PAGE_MARKER = re.compile(r"^--- 第\d+页 ---$")

def title_hint(text: str, max_len: int = 30) -> str:
    """给提炼当兜底标题：第一条正文行，跳过我们自己插的分页标记。"""
    for line in text.splitlines():
        line = line.strip()
        if line and not _PAGE_MARKER.match(line):
            return line[:max_len]
    return ""

# ===== backend/app/services/poster_recipe.py — 卡片配方服务 =====
"""配方校验（方案 docs/方案-卡片模板不走发版.md §三 第 3 层那道闸门的服务端半边）。

客户端那份 `posterRecipe.js` 里有一个 `validate()`，这里就是它的 Python 对照：
名单外的一切（op、字段、原语、参数名、算子、步型、令牌名）一律不放行。
**下发前拦一道、客户端渲染前再拦一道**，两道都拦是因为它们会漂移——客户端在用户手机上，
它的名单跟着包版本走；服务器只有一份，写的却是所有版本都要读的东西。

名单为什么在这里抄第二份而不是直接读 `backend/seed/poster_whitelist.json`：
运行期不该依赖一个"要跑一次 node 才生成得出"的文件。漂移由两条检查钉住：
- `backend/tests/test_poster_templates.py` 第一条：这份 Python 名单 == 种子 JSON（两边逐键比）；
- `docs/工具/验-模板配方可执行.js` 里那条：种子 JSON == 现读 `poster.js` 算出来的那份。
两条都在，中间就没有"谁也没看的那一段"。

还有一条上界是真要命的：解释器对"配方有多大"不设防，一条十万步的配方会让出图那一次
主线程卡在测量里。`LIMITS` 那三个数就是给这个的，与客户端那份必须同值——
同上，由 pytest 与那把尺子两头夹。
"""
import json
from typing import Any

# ---------------------------------------------------------------- 闭集名单（与 poster.js /
posterRecipe.js 同值）

ARITH: dict[str, int] = {
    "+": -1, "-": -1, "*": -1, "/": 2, "%": 2,
    "min": -1, "max": -1, "round": 1, "floor": 1, "ceil": 1, "abs": 1, "neg": 1,
}
LOGIC: dict[str, int] = {
    "<": 2, "<=": 2, ">": 2, ">=": 2, "==": 2, "!=": 2,
    "and": -1, "or": -1, "not": 1, "len": 1, "truthy": 1,
}
CONV: dict[str, int] = {"str": 1, "padStart": 2, "slice": 3}
STEPS = ("let", "do", "emit", "emitOne", "emitMany", "if", "each")
TERM_FORMS = ("var", "lit", "if", "obj", "prim", "list")

# `emit` 里那些图层对象的字段名，按 op 分组。op 名单本身也从这里读。
OP_KEYS: dict[str, tuple[str, ...]] = {
    "fill": ("x", "y", "w", "h", "color"),
    "grad": ("x", "y", "w", "h", "c1", "c2", "dir", "stops"),
    "radial": ("x", "y", "r0", "r1", "c1", "c2", "box"),
    "rrect": ("x", "y", "w", "h", "r", "fill", "shadow", "shadowBlur", "shadowY", "stroke",
"strokeWidth"),
    "circle": ("x", "y", "r", "fill", "stroke", "strokeWidth"),
    "line": ("x1", "y1", "x2", "y2", "w", "color"),
    "dots": ("x", "y", "w", "h", "gap", "r", "color", "oddRowShift"),
    "text": ("x", "y", "lines", "lh", "size", "weight", "color", "align", "fam",
             "alpha", "track", "stroke", "strokeWidth", "vert", "colGap", "vpunct"),
```

```
    "image": ("key", "x", "y", "w", "h", "placeholder", "r", "clipCircle", "gray", "tint",
"fadeFrom"),
    "avatar": ("x", "y", "d", "ring", "ringColor", "fallback"),
}
PRIM_KEYS: dict[str, tuple[str, ...]] = {
    "i18n": ("key",),
    "blockName": (),
    "sourceLabel": (),
    "quoteText": (),
    "noteDate": (),
    "tagsJoined": ("sep",),
    "paperOf": ("categoryId",),
    "scheme": ("name", "categoryId"),
    "plateColors": ("categoryId",),
    "withAlpha": ("color", "alpha"),
    "mix": ("c1", "c2", "w"),
    "joinNonEmpty": ("sep", "parts"),
    "points": ("limit", "maxW", "size", "bold", "fam"),
    "fitLines": ("text", "maxW", "n", "size", "bold", "fam"),
    "wrapLines": ("text", "maxW", "size", "bold", "fam"),
    "clipLine": ("text", "maxW", "size", "bold", "fam"),
    "clipTrack": ("text", "maxW", "track", "size", "bold", "fam"),
    "trackWidth": ("text", "track", "size", "bold", "fam"),
    "fitSize": ("text", "maxW", "want", "floor"),
    "setFont": ("size", "bold", "fam"),
    "vertCols": ("text", "colH", "step", "maxCols", "size", "bold", "fam"),
    "vertAdv": ("text", "step", "size", "bold", "fam"),
    "maxOf": ("values", "seed"),
    "brandGlyph": (),
    "signRow": ("x", "y", "maxW", "size", "avatarD", "onDark", "hasAvatar"),
    "glyphPlate": ("x", "y", "w", "h", "color"),
    "qrSticker": ("x", "y", "size", "offset", "ink", "label"),
    "qrStickerH": ("size",),
    "footH": ("qrSize",),
    "token": ("name",),
}
TOKENS = ("paper", "ink", "body", "muted", "hard", "warm", "serif", "mono")

# 包内十套实测峰值：最大 6,383 字节（card）、最深 17 层（spec）、最多 691 个节点（card）。
# 这三条留了两三倍余量。最终数值是待拍项（方案 §五），但**两处必须相等**这条现在就能钉：

# 客户端那份 LIMITS 是唯一出处，种子只是搬运，pytest 与那把尺子各查一头。
LIMITS = {"max_bytes": 16384, "max_depth": 24, "max_nodes": 2000}
INTERPRETER_VERSION = 1

_ARITY = {**ARITH, **LOGIC, **CONV}

# ---------------------------------------------------------------- 形状

def _shape(t: Any):
    """返回 (种类, 内容) 或 (None, 错误话)。与 JS 那份 shape() 一一对应。"""
    if t is None or isinstance(t, (bool, int, float, str)):
        return "lit", t
    if not isinstance(t, (dict, list)):
        return None, f"不成形的值（{type(t).__name__}）"
    if isinstance(t, list):
        return "list", t
    if "prim" in t:
        if not isinstance(t["prim"], str):
            return None, "prim 得是个名字"
        extra = [k for k in t if k not in ("prim", "args")]
        if extra:
            return None, f"原语那一项里多了 {'、'.join(extra)}"
        args = t.get("args", {})
```

```
        if not isinstance(args, dict):
            return None, f"原语「{t['prim']}」的 args 得是个对象"
        return "prim", (t["prim"], args)
    if "obj" in t:
        extra = [k for k in t if k != "obj"]
        if extra:
            return None, f"obj 那一项里多了 {'、'.join(extra)}"
        if not isinstance(t["obj"], dict):
            return None, "obj 得是个键值对象"
        return "obj", t["obj"]
    if len(t) != 1:
        return None, f"只能有一个键，这里有 {len(t)} 个：{'、'.join(t)}"
    return "op", next(iter(t.items()))

def _step_shape(raw: Any):
    if not isinstance(raw, dict):
        return None, "一步得是个对象"
    if "let" in raw:
        extra = [k for k in raw if k not in ("let", "value")]
        if extra:
            return None, f"let 那一步里多了 {'、'.join(extra)}"
        if not isinstance(raw["let"], str) or not raw["let"]:
            return None, "let 要给一个变量名"
        if "value" not in raw:
            return None, f"let「{raw['let']}」少了 value"
        return "let", (raw["let"], raw["value"])
    if len(raw) != 1:

        return None, f"一步只能有一个键，这里有 {len(raw)} 个：{'、'.join(raw)}"
    kind = next(iter(raw))
    if kind not in STEPS:
        return None, f"不认的步型「{kind}」，能用的是 {'、'.join(STEPS)}"
    return kind, raw[kind]

# ---------------------------------------------------------------- 大小与深度

def canonical_json(v: Any) -> str:
    """按键名排序的紧凑 JSON。字节数上限按这个算，不然同一份配方会因为键序算出两个大小。"""
    return json.dumps(v, ensure_ascii=False, sort_keys=True, separators=(",", ":"))

def _depth(v: Any) -> int:
    if isinstance(v, list):
        return 1 + max([_depth(x) for x in v], default=0)
    if isinstance(v, dict):
        return 1 + max([_depth(x) for x in v.values()], default=0)
    return 0

def _nodes(v: Any) -> int:
    if isinstance(v, list):
        return len(v) + sum(_nodes(x) for x in v)
    if isinstance(v, dict):
        return len(v) + sum(_nodes(x) for x in v.values())
    return 0

# ---------------------------------------------------------------- 静态检查

def _scan_term(t: Any, bad: list, where: str) -> None:
    kind, payload = _shape(t)
    if kind is None:
        bad.append(f"{where}: {payload}")
        return
    if kind == "lit":
        return
    if kind == "list":
```

```
        for i, x in enumerate(payload):
            _scan_term(x, bad, f"{where}[{i}]")
        return
    if kind == "prim":
        name, args = payload
        keys = PRIM_KEYS.get(name)
        if keys is None:
            bad.append(f"{where}: 原语「{name}」不在名单里")
            return
        for k in args:
            if k not in keys:
                bad.append(f"{where}: 原语「{name}」没有参数「{k}」，能用的是 {'、'.join(keys)}")
        for k in args:
            _scan_term(args[k], bad, f"{where} {name}.{k}")

        return
    if kind == "obj":
        for k, v in payload.items():
            _scan_term(v, bad, f"{where} obj.{k}")
        return
    name, a = payload
    if name in ("var", "lit"):
        if name == "var" and (not isinstance(a, str) or not a):
            bad.append(f"{where}: var 得是个名字")
        return
    if name == "if":
        if not isinstance(a, list) or len(a) != 3:
            bad.append(f"{where}: if 要 [条件, 真, 假] 三个参数")
            return
        for i, x in enumerate(a):
            _scan_term(x, bad, f"{where} if[{i}]")
        return
    want = _ARITY.get(name)
    if want is not None:
        args = a if isinstance(a, list) else [a]
        if want >= 0 and len(args) != want:
            bad.append(f"{where}: 算子「{name}」要 {want} 个参数，给了 {len(args)} 个")
        if want < 0 and len(args) < 1:
            bad.append(f"{where}: 算子「{name}」至少要一个参数")
        for i, x in enumerate(args):
            _scan_term(x, bad, f"{where} {name}[{i}]")
        return
    bad.append(f"{where}: 不认的算子「{name}」")

def _scan_steps(steps: Any, bad: list, where: str) -> None:
    if not isinstance(steps, list):
        bad.append(f"{where}: steps 得是个数组")
        return
    for i, raw in enumerate(steps):
        at = f"{where} 第 {i + 1} 步"
        kind, payload = _step_shape(raw)
        if kind is None:
            bad.append(f"{at}: {payload}")
            continue
        if kind == "let":
            _scan_term(payload[1], bad, f"{at} {payload[0]}")
            continue
        if kind == "do":
            _scan_term(payload, bad, f"{at} do")
            continue
        if kind == "emit":
            a = payload
            if not isinstance(a, dict):
                bad.append(f"{at}: emit 要给一个图层对象")
```

```
                continue

            op = a.get("k")
            fields = OP_KEYS.get(op if isinstance(op, str) else "")
            if fields is None:
                bad.append(f"{at}: 绘制 op「{op if op is not None else '（没写 k）'}」不在名单里")
                continue
            for k in a:
                if k == "k":
                    continue
                if k not in fields:
                    bad.append(f"{at}: op「{op}」没有字段「{k}」，能用的是 {'、'.join(fields)}")
                    continue
                _scan_term(a[k], bad, f"{at} {op}.{k}")
            continue
        if kind in ("emitOne", "emitMany"):
            _scan_term(payload, bad, f"{at} {kind}")
            continue
        if kind == "if":
            a = payload
            if not isinstance(a, dict):
                bad.append(f"{at}: if 要给 {{cond, then, else}}")
                continue
            _scan_term(a.get("cond"), bad, f"{at} if.cond")
            _scan_steps(a.get("then"), bad, f"{at} if.then")
            if a.get("else") is not None:
                _scan_steps(a["else"], bad, f"{at} if.else")
            for k in a:
                if k not in ("cond", "then", "else"):
                    bad.append(f"{at}: if 项里多了 {k}")
            continue
        # 只剩 each 这一步型了（STEPS 已在 _step_shape 里挡过一遍）
        a = payload
        if not isinstance(a, dict):
            bad.append(f"{at}: each 要给 {{over, as, index, do}}")
            continue
        _scan_term(a.get("over"), bad, f"{at} each.over")
        if not isinstance(a.get("as"), str) or not a["as"]:
            bad.append(f"{at}: each 少了 as")
        _scan_steps(a.get("do"), bad, f"{at} each.do")
        for k in a:
            if k not in ("over", "as", "index", "do"):
                bad.append(f"{at}: each 项里多了 {k}")

def validate_recipe(recipe: Any) -> list[str]:
    """返回错误清单，空列表＝这份配方这一版解释器读得懂。写回前、下发前都过它。

    和客户端那条"整条配方一起丢"的规则是一对：这里放行了而客户端读不懂，
    结果不是少一层，而是整条退回包内那一份——所以宁可在这里就报给写配方的人。
    """
    if not isinstance(recipe, dict):
        return ["配方得是个 JSON 对象"]

    bad: list[str] = []
    if not isinstance(recipe.get("id"), str) or not recipe["id"]:
        bad.append("少了 id")
    mv = recipe.get("min_version")
    if not isinstance(mv, int) or isinstance(mv, bool) or mv < 1:
        bad.append("min_version 得是 ≥1 的整数")
    elif mv > INTERPRETER_VERSION:
        bad.append(f"这份配方要 {mv} 版解释器，现在这份只会读到 {INTERPRETER_VERSION} 版")
    steps = recipe.get("steps")
    if not isinstance(steps, list) or not steps:
        bad.append("steps 得是非空数组")
```

```
    else:
        _scan_steps(steps, bad, "steps")
        if not any(isinstance(s, dict) and s.get("let") == "height" for s in steps):
            bad.append("steps 里必须有一步 let 出 height")
    n_bytes = len(canonical_json(recipe).encode("utf-8"))
    if n_bytes > LIMITS["max_bytes"]:
        bad.append(f"整条配方 {n_bytes} 字节，超过上限 {LIMITS['max_bytes']}")
    depth = _depth(recipe)
    if depth > LIMITS["max_depth"]:
        bad.append(f"嵌套 {depth} 层，超过上限 {LIMITS['max_depth']}")
    nodes = _nodes(recipe)
    if nodes > LIMITS["max_nodes"]:
        bad.append(f"{nodes} 个节点，超过上限 {LIMITS['max_nodes']}")
    return bad

def recipe_whitelist() -> dict:
    """这份 Python 名单的导出形态，键名与 docs/工具/出-模板配方种子.js 对齐，给 pytest 比。"""
    return {
        "interpreter_version": INTERPRETER_VERSION,
        "steps": list(STEPS),
        "arith": ARITH,
        "logic": LOGIC,
        "conv": CONV,
        "term_forms": list(TERM_FORMS),
        "op_keys": {k: list(v) for k, v in OP_KEYS.items()},
        "prim_keys": {k: list(v) for k, v in PRIM_KEYS.items()},
        "tokens": list(TOKENS),
        "limits": dict(LIMITS),
    }

# ===== backend/app/services/poster_templates.py — 模板管理服务 =====
"""模板表的两件事：把包内那十套幂等灌进去（种子），以及按状态读出来（下发）。

**写路径只有这一个函数，没有 HTTP 写接口**。这不是偷懒，是这一批的边界（方案 §七 负面清单）：
"不走发版"要的是站长改一条记录就能看到新卡片，不是给外部一个能改画面的口子。
真要改的时候是：改 `miniprogram/utils/posterRecipes.js`（或直接在库里改一行）→ 跑那两把尺子 →
重新生成种子 → 跑一次这个函数。方案 P1 那一步"服务端加第 11 套"也是走这一条，不新增接口。
"""
import hashlib

import json
import os

from sqlalchemy.orm import Session

from app.models.poster_template import GROUP_KEYS, STATUS_LIVE, PosterTemplate
from app.services.poster_recipe import canonical_json, validate_recipe

SEED_PATH = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), "seed",
"poster_templates.json")

def seed_rows(path: str = SEED_PATH):
    with open(path, encoding="utf-8") as f:
        return json.load(f)

def seed_poster_templates(db: Session, path: str = SEED_PATH) -> dict:
    """幂等灌种子：同 template_id + 同 content_hash 的行不动，内容变了就新增一版并把旧的收档。

    为什么"新增一版"而不是原地 UPDATE：已经下发出去的那份配方，可能正躺在某个用户的本地缓存里。
    原地改会让"同一个 hash 对应两种内容"，那条缓存判据（客户端号没变就不重解析）就成了假的。
    收档的老行留着不删，删行等于把历史抹掉。
    """
    report = {"inserted": 0, "unchanged": 0, "archived": 0, "rejected": []}
    for row in seed_rows(path):
```

```
        errors = validate_recipe(row["recipe"])
        if errors:
            # 脏种子一条都不落库：落半份等于把"这张卡画不出来"推给用户去兜。
            report["rejected"].append(f"{row['template_id']}: {'; '.join(errors)}")
            continue
        if row["group_key"] not in GROUP_KEYS:
            report["rejected"].append(f"{row['template_id']}: group_key「{row['group_key']}」不在名
单里")
            continue
        # 配方里那个 id 必须就是这一行的 template_id——与客户端 poster.js 的 remoteRowProblems
        # 同一条规则（两边各拦一遍，服务端这道先拦，写配方的人当场就能看到）。
        # 少这一道，"加第 11 套"可以写成把 quote 那份配方贴到新 id 上：每一层都合法、
        # 客户端也收，但列表里多那一格，点进去画的是另一张卡——名字对、内容不对。
        if (row["recipe"] or {}).get("id") != row["template_id"]:
            report["rejected"].append(
                f"{row['template_id']}: 配方自己的 id「{(row['recipe'] or {}).get('id')}」与
template_id 不一致"
            )
            continue
        existing = (
            db.query(PosterTemplate)
            .filter(
                PosterTemplate.template_id == row["template_id"],
                PosterTemplate.status == STATUS_LIVE,
            )
            .first()
        )
        if existing and existing.content_hash == row["content_hash"]:
            report["unchanged"] += 1

            continue
        if existing:
            existing.status = "archived"
            report["archived"] += 1
        db.add(
            PosterTemplate(
                template_id=row["template_id"],
                label=row["label"],
                label_en=row["label_en"],
                group_key=row["group_key"],
                sort_order=row["sort_order"],
                recipe=row["recipe"],
                status=row.get("status", STATUS_LIVE),
                min_app_version=row.get("min_app_version", "1.0.0"),
                content_hash=row["content_hash"],
                note=row.get("note"),
            )
        )
        report["inserted"] += 1
    db.commit()
    return report

def live_templates(db: Session):
    """只读 live 那几行。draft（还在改的）与 archived（收回去的）都不出场——
    回滚就是改这一个 status 字段，秒级生效，不发版。"""
    return (
        db.query(PosterTemplate)
        .filter(PosterTemplate.status == STATUS_LIVE)
        .order_by(PosterTemplate.sort_order.asc(), PosterTemplate.id.asc())
        .all()
    )

def content_hash_of(recipe) -> str:
    """与 docs/工具/出-模板配方种子.js 那个 hash 同一条口径：键名排序的紧凑 JSON 取 sha256。
```

```
    两边算出的串必须一致（test_poster_templates 里钉着），否则客户端"号没变就不重解析"
    那条缓存判据会因为一次服务端重算而全线失效——那种失效是静默的，看起来只是缓存不命中。
    """
    return hashlib.sha256(canonical_json(recipe).encode("utf-8")).hexdigest()

# ===== backend/app/services/quota.py — 容量配额服务 =====
"""MIND（积分）与邀请台账：规则只写在这一个文件里，路由和 worker 都只调这里的函数。

口径（2026-10-01 站长改定，原话"拆掉后端 100 篇，不同用户转存都加分，但同一个用户只算一次。
其他的照常。积分规则，可以在官网公布，小程序不需要"）：
- **笔记不限量**。原来那道"每人 100 篇、第 101 篇四条入口当场拦下"的闸门撤掉了，
  100 变成 MIND 的起始值，不再是一个上限；
- 带来一个**新的写作者**给邀请人 **+10**：他动笔写下自己名下第一条才算，
  一个人一辈子只成就一次（`source_note_id IS NULL` 那半边唯一索引）；
- **有人把某一篇转存进自己库里**给原作者 **+1**：不同用户各算一次，

  同一个人对同一篇只算一次（`source_note_id` 非空那半边按"这篇 × 这个人"唯一）。
  转存这条不再要求对方是新用户——老用户转存也算。

能挡的是"同一个人对同一篇反复转存刷分"，挡不掉的是注册小号互转（微信 openid 人手一个），
所以不设总闸、也不设次数上限，跟 +10 那条一个 posture。

起始值、两档奖励都只写在这里，客户端不硬编码：它读 GET /api/user/quota 拿回来的 mind。
"""
import logging

from sqlalchemy import update
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.models.invitation import Invitation
from app.models.note import Note
from app.models.user import User

logger = logging.getLogger(__name__)

BASE_QUOTA = 100        # MIND 的起始值（不再是"每人几篇"的上限）
INVITE_REWARD = 10      # 带来一个新的写作者
IMPORT_REWARD = 1       # 有人把某一篇转存进自己库里

def mind_score(user: User) -> int:
    """这一页右下角那个大数字。"""
    return BASE_QUOTA + int(user.quota_bonus or 0)

def used_count(db: Session, user: User) -> int:
    return db.query(Note).filter(Note.user_id == str(user.id)).count()

def rewarded_invites(db: Session, inviter_id: int) -> int:
    """"这个人替我带来了几个新写作者"——只数动笔那一路。

    2026-10-01 加了转存 +1 之后，台账里混着两种行；不加这个过滤，字段名就成了假话
    （别人转存一次也会被算成"带来一个人"）。
    """
    return (
        db.query(Invitation)
        .filter(Invitation.inviter_id == inviter_id, Invitation.source_note_id.is_(None))
        .count()
    )

def quota_view(user: User, db: Session) -> dict:
    """`limit` / `remaining` 是留给线上老客户端的：1.5.0 那一版「我的」读的是
    `${used}/${limit}`，字段删了那一行会显示成 "37/undefined"。闸门已经撤了，
    这两个数不再拦任何人，等线上换到读 `mind` 的版本之后就可以一起删掉。
```

```
    """
    used = used_count(db, user)
    mind = mind_score(user)

    rewarded = rewarded_invites(db, user.id)
    from app.models.category import Category

    return {
        "used": used,
        "categories": db.query(Category).filter(Category.user_id == str(user.id)).count(),
        "mind": mind,
        "limit": mind,
        "remaining": max(0, mind - used),
        "base": BASE_QUOTA,
        "bonus": int(user.quota_bonus or 0),
        "reward_each": INVITE_REWARD,
        "import_each": IMPORT_REWARD,
        "invites_rewarded": rewarded,
    }

def attribute_inviter(user: User, db: Session, inviter_id) -> bool:
    """登录时把"谁邀我来的"记在账号上；只记账，不发奖励。

    四条不认：没带参数、自己邀自己、已经有归属、名下已经有笔记。
    最后一条是给老用户的——他哪天从别人的分享链接进来一次，不能就此把归属改写到
    那个人名下，那笔奖励就凭空冒出来了。
    """
    try:
        inviter_pk = int(inviter_id)
    except (TypeError, ValueError):
        return False
    if inviter_pk <= 0 or inviter_pk == user.id or user.invited_by is not None:
        return False
    if db.get(User, inviter_pk) is None:
        return False
    if used_count(db, user) > 0:
        return False
    user.invited_by = inviter_pk
    db.commit()
    logger.info("邀请归因：invitee=%s inviter=%s", user.id, inviter_pk)
    return True

def _is_his_first_note(db: Session, user: User) -> bool:
    """刚 commit 完这条，他名下正好一条 → 这就是他动笔写下的第一篇。"""
    return used_count(db, user) == 1

def _settle(db: Session, inviter: User, invitee: User, source_note_id, reward: int, why: str) ->
int:
    """两条路共用的那一段：记一笔台账，给收账的人加这一档分。

    唯一索引是最后一道闸。两个请求真同时进来时（同步路由跑在线程池里，确实会并发），
    后到的那个会在 commit 撞上 IntegrityError，由公开入口那层 `_never_breaks` 兜住：
    回滚的只有这一笔台账和这一笔加分，笔记本身早就 commit 过了，
    所以用户那边仍然只看见"存好了"。这里不留 except——留了就成了两层一样的兜底，
    而变异检查会告诉你其中一层删掉行为不变（那就是没在测东西）。


    余额走 SQL 侧自增，不在 Python 里"读出来加完写回去"：那样两笔并发各自读到旧值、
    后写的把先写的覆盖掉，台账记了 6 行而余额只加 1（临时库 6 线程真并发实测过）。
    自增和 INSERT 在同一个事务里，撞索引时一起回滚，不会留下"分了加了、账没记"。
    """
    db.add(
        Invitation(
            inviter_id=inviter.id,
```

```
            invitee_id=invitee.id,
            source_note_id=source_note_id,
            reward=reward,
        )
    )
    db.execute(
        update(User).where(User.id == inviter.id).values(quota_bonus=User.quota_bonus + reward)
    )
    db.commit()
    logger.info(
        "MIND 到账（%s）：收账人=%s +%d（现 %d）来人=%s 源笔记=%s",
        why, inviter.id, reward, mind_score(inviter), invitee.id, source_note_id,
    )
    return reward

def _credit_first_note(note: Note, db: Session, user: User) -> int:
    """被邀请人自己动笔写下第一篇笔记 → 给邀请人 +10。返回本次实际到账分（0 = 没到账）。

    在笔记 commit 之后调用。一个人只成就一次这件事由数据库兜：动笔那一路
    `source_note_id` 留空，`invitee_id` 上的部分唯一索引（只钉空行）保证同一个人
    第二次动笔也结不出第二笔。
    """
    if not user.invited_by or not _is_his_first_note(db, user):
        return 0

    inviter = db.get(User, user.invited_by)
    if inviter is None or inviter.id == user.id:
        # 自己指向自己这种状态，登录那条路（attribute_inviter）本来就写不出来；
        # 这里再挡一次是因为结钱的判定不该依赖"上游不会漏"。
        return 0
    return _settle(db, inviter, user, None, INVITE_REWARD, "动笔首篇")

def _credit_import(db: Session, importer: User, source_note_id) -> int:
    """有人把某一篇转存进自己库里 → 给那篇的作者 +1。

    口径（2026-10-01 改定）：**不同用户各算一次，同一个用户对同一篇只算一次**。
    不再要求转存的人是新用户——老用户转存也算，所以原来那句"他名下正好一条"
    从这条路上撤掉了。防线从"一篇一辈子一次"换成"一篇 × 一个人一次"：
    `source_note_id` 非空那半边的复合唯一索引。

    挡得住同一个人反复转存刷分，挡不住注册小号互转（微信 openid 人手一个）——
    这一条和 +10 那条是同一个 posture，不设总闸也不设上限。

    """
    try:
        src = db.get(Note, int(source_note_id))
    except (TypeError, ValueError):
        return 0
    if src is None:
        return 0

    try:
        author_pk = int(src.user_id)
    except (TypeError, ValueError):
        return 0
    if author_pk == importer.id:
        # 自己转自己那篇不算：转存入口本来就只给别人看的那篇才出现，
        # 但接口能直接被调，所以这一刀在这里补上。
        return 0
    inviter = db.get(User, author_pk)
    if inviter is None:
        return 0
    already = (
        db.query(Invitation)
```

```
        .filter(Invitation.source_note_id == src.id, Invitation.invitee_id == importer.id)
        .first()
    )
    if already is not None:
        # 索引也会挡，先查一次是为了把日志说清楚：这是"同一个人第二次转同一篇"，
        # 不是并发撞车。
        logger.info("这个人已经因为这篇得过分，未到账：源笔记=%s 作者=%s 来人=%s", src.id, inviter.id,
importer.id)
        return 0
    return _settle(db, inviter, importer, src.id, IMPORT_REWARD, "转存")

def credit_first_note(note: Note, db: Session, user: User) -> int:
    """公开入口：结不到账绝不能把一个已经存好的笔记回成 500。判据在 `_credit_first_note`。"""
    return _never_breaks(db, "动笔首篇", _credit_first_note, note, db, user)

def credit_import(db: Session, importer: User, source_note_id) -> int:
    """公开入口：同上，判据在 `_credit_import`。"""
    return _never_breaks(db, "转存", _credit_import, db, importer, source_note_id)

def _never_breaks(db: Session, what: str, fn, *args) -> int:
    """只兜 `IntegrityError` 是不够的。

    API 与 worker 共用同一个 SQLite 文件，`database is locked` 是 OperationalError：
    那时候笔记已经 commit 过了，让它冒到路由上就是回 500——用户重试会多存一份，
    而那笔分因为名下条数变了再也结不出来。所以这里把一切异常都收在"这笔没结成"，
    原文只进日志。**笔记存好了这件事，比分数到没到账重要。**
    """
    try:
        return fn(*args)
    except IntegrityError:

        db.rollback()
        logger.info("这一笔已被并发的那趟记走，未重复到账：%s", what)
        return 0
    except Exception:
        db.rollback()
        logger.exception("MIND 结算失败（笔记已保存，不影响用户）：%s", what)
        return 0

# ===== backend/app/services/sharing.py — 分享服务 =====
"""分享这条链上"公开可见"的口径，收在这一个文件里。

三份定义被创建分享和编辑笔记两条路共用，而它们必须是同一份：
- `SNAPSHOT_COLUMNS`：公开页真正显示的字段。笔记里这几列一变，快照就得跟着同步，
  并且要重新过一次公开那道闸。
- `public_fields`：会因为一次分享而对外可见的全部内容，也就是送检范围。
- `sync_snapshot` / `snapshot_matches`：快照是冗余存的，公开页读的是它而不是笔记本身。
  笔记一改就得跟着同步，否则用户把隐私内容改掉之后，已经发出去的那张卡片仍然挂着旧内容。
- `close_shares`：公开/不公开只由 `shares.is_active` 这一列说话。海报上的码不设过期，
  所以"关掉这扇门"是用户能做的唯一下线动作——它必须存在，否则发出去就再也收不回来。

字段清单曾经漏掉过 key_links：它客户端可写、又显示在公开页上，却一次都没进过
msgSecCheck。所以下面这几个函数都从常量取字段，不再各写一份元组。
"""
from datetime import datetime, timezone

from datetime import datetime, timezone

from sqlalchemy.orm import Session

from app.models.note import Note
from app.models.share import Share

# 公开页上真正显示的那几列（= ShareResponse 除 token/created_at 之外的全部字段）。
```

```
# 原来只有 title/summary/tags/key_links，缺了 key_points 和 source_url——扫码的人
# 只能看到一段摘要，既读不到要点，也拿不到原文链接，观感上就是"看不到原文"。
SNAPSHOT_COLUMNS = ("title", "summary", "tags", "key_links", "key_points", "source_url")

# 公开页上显示、但**不是**从笔记搬过来的字段。目前只有分享者昵称：它来自「分享形象」，
# 不进 SNAPSHOT_COLUMNS（那几列会被 sync_snapshot 从笔记覆盖，昵称不该被笔记内容带跑），
# 但它和快照字段一样是"因为这一次分享才对外可见"的内容，所以必须进送检。
# 加这一列而不是一句注释，是为了让用例能拿它和 ShareResponse 的字段集对账。
NON_NOTE_PUBLIC_COLUMNS = ("author_name",)

def public_fields(note: Note) -> tuple:
    """建分享时送检的全部字段。顺序即送检顺序，创建与编辑两条路共用这一份。

    必须**覆盖** `SNAPSHOT_COLUMNS`：公开页会显示而没送检的字段，就是一条绕过内容安全
    直接上公开页的路（key_links 当初就是这么漏的）。正文和外部原文不在公开页上，
    但建分享时一并看住更稳妥——多检不漏检。
    """

    return (
        note.title,
        note.summary,
        note.tags,
        note.key_points,
        note.key_links,
        note.source_url,
        note.content,
        note.original_content,
    )

def public_check_fields(note: Note, author_name: str | None = None) -> tuple:
    """建分享这一次真正送检的全部字段：笔记那几列 + 分享者昵称。

    公开页上出现而没进这道闸的字段，就是一条绕过内容安全直接上公开页的路
    （key_links 当初就是这么漏的）。所以新增 NON_NOTE_PUBLIC_COLUMNS 里的任何一列，
    都必须从这里出去，而不是在调用方各写一份参数列表。
    """
    return (*public_fields(note), author_name)

def visible_fields(note: Note) -> dict:
    """快照里真正会出现在公开页上的那几列（ShareResponse 的字段集）。"""
    return {col: getattr(note, col) for col in SNAPSHOT_COLUMNS}

def active_shares(db: Session, note_id: int) -> list[Share]:
    return (
        db.query(Share)
        .filter(Share.note_id == note_id, Share.is_active == True)  # noqa: E712
        .all()
    )

def snapshot_matches(share: Share, note: Note) -> bool:
    """公开页上那几列是否已经和笔记当前内容一模一样。

    只比 visible_fields 而不是整条笔记：正文变了但公开页看不出变化时，重复建分享
    确实没必要再打一次 msgSecCheck。
    """
    return all(getattr(share, col) == value for col, value in visible_fields(note).items())

def sync_snapshot(db: Session, note: Note) -> int:
    """把这条笔记名下所有仍然有效的分享快照刷成当前内容，返回刷了几条。

    不 commit——调用方决定这一笔和它自己的改动是一次提交。编辑笔记时如果送检
    没过，回滚要连这次同步一起回滚，否则会出现"公开页拿到了没过审的新内容"。
    """
    shares = active_shares(db, note.id)
```

```
    for share in shares:
        for column, value in visible_fields(note).items():
            setattr(share, column, value)
    if shares:

        db.flush()
    return len(shares)

def close_shares(db: Session, note_id: int) -> int:
    """把这篇笔记名下还开着的分享全关掉，返回关掉几条。撤回分享走的就是这里。

    关而不删：token 留着，哪张海报上的码被谁扫过还能对账（回 404"分享已关闭"），
    而 is_active 是唯一那扇"用户愿不愿意让别人看"的门——有效期已经不当这个用了。
    不 commit，和调用方的其它改动同一次提交。
    """
    return (
        db.query(Share)
        .filter(Share.note_id == note_id, Share.is_active == True)  # noqa: E712
        .update({"is_active": False}, synchronize_session=False)
    )

def is_expired(share: Share) -> bool:
    """这张码按它自己那个过期时间算死了没有。

    只有上线前建的那批带 `expires_at`，新建的一律为 None（= 不过期）。判过期时间的
    口径必须只有一份：库里存过 '…+00:00' 和不带时区两种写法，不带时区的按 UTC 读，
    和公开页那侧的行为一致。
    """
    if not share.expires_at:
        return False
    expires = share.expires_at
    if expires.tzinfo is None:
        expires = expires.replace(tzinfo=timezone.utc)
    return datetime.now(timezone.utc) >= expires

def active_share_by_token(db: Session, token: str) -> Share | None:
    """按 token 找到那张**此刻真的还对外开着**的码，找不到或已过期都返回 None。

    公开页读分享、把分享转存成别人的笔记，两条路问的是同一个问题，所以这个判断
    只能有一个出处——否则很容易出现"落地页显示 404 但转存接口照样抄走内容"。
    """
    share = db.query(Share).filter(Share.token == token, Share.is_active == True).first()  # noqa:
E712
    if not share or is_expired(share):
        return None
    return share

# ===== backend/app/services/wechat.py — 微信 API 对接 =====
import hashlib
import json
import logging
import time
from collections import OrderedDict

import httpx


from app.core.config import settings
from app.core.errors import UserError

logger = logging.getLogger(__name__)

# 不用 /cgi-bin/token：那个接口是"轮转"的，任何第二个进程（本地调试、另一台机器、
# 甚至同项目的 worker）拿一次凭据去要 token，就会把前一个进程手里那份挤成
```

```
# "invalid credential / not latest"（40001），而本进程的缓存还要抱着旧 token 到过期为止。
# 2026-09-24 现网就被这么打断过：二维码和内容安全两条一起 502/降级。
# 稳定版接口按凭据回同一份"当前有效"的 token，不轮转，所以多进程不再互相踢。
WX_STABLE_TOKEN_URL = "https://api.weixin.qq.com/cgi-bin/stable_token"
WX_QRCODE_URL = "https://api.weixin.qq.com/wxa/getwxacodeunlimit"
WX_SEC_CHECK_URL = "https://api.weixin.qq.com/wxa/msg_sec_check"
# 这三种都是"token 这一环不对"，不是我们送的内容有问题：清缓存重取一次再判。
TOKEN_ERRORS = {40001, 42001, 40014}

# 实测（2026-09-22，现网凭据）：v2 送 2,501 字和 6,000 字都正常返回，不报错。
# 但文档口径是单次 2,500 字符，而且模型到底看了前 2,500 还是全文无从验证——
# 分段送才不漏检，所以按 2,000 字一段切。
SEC_CHUNK = 2000
# 一段最多送几次：8 段 × 2,000 = 16,000 字，这就是**单个字段能被检到的上限**。
# notes 路由的 MAX_BODY / MAX_SUMMARY 特意钉在同一个数上——手打内容一旦超过这个窗口，
# 多出来的部分永远不会被检，把违规文本垫在一万六千字之后就是一条现成的绕过路径。
# 抓取来的正文可以远超这个窗口（worker 直接落库），那部分只能检前 16,000 字，
# 所以下面截断时会记一条 warning，别让"检了"和"其实只检了开头"混成一件事。
SEC_MAX_CHUNKS = 8

_access_token_cache = {"token": None, "expires_at": 0}

# 小程序码是"同一个 scene 出一张一模一样的图"，每次都去微信要等于白烧那个接口的日配额，
# 而且从我们这边的错误率完全看不出来（微信回得一切正常，钱却花掉了）。
# 上限是刻意的：一张码实测约 55KB，100 张封顶 5.5MB，这台机器内存本来就紧。
# 键里带 env_version，所以测试期改成 trial、发布前改回 release 时，旧图自然失效。
_QR_CACHE_MAX = 100
_qr_cache: "OrderedDict[tuple, bytes]" = OrderedDict()

# 内容安全是**按次配额**的接口：未上架的小程序 100 次/天（实测 2026-09-23 打到过上限，
# 回 45009），上架后才是 200 万次/天。一次保存要按字段打好几趟，而其中绝大多数是
# "这段文本今天已经判过正常"——所以按 (openid, 文本散列) 记住 pass。
# 只记 pass：risky/review 不进缓存，判罚偶尔不对时下一次还能纠正；unavailable 更进不得，
# 那等于把"配额打光"固化成永久放行，第二天额度恢复了也不会再检。
_SEC_PASS_CACHE_MAX = 4000
_sec_pass_cache: "OrderedDict[tuple, bool]" = OrderedDict()

def _invalidate_access_token() -> None:
    """丢掉缓存的 token。拿到 40001 这一类"凭据不对"时必须调，否则旧值会一直抱到过期。"""
    _access_token_cache["token"] = None
    _access_token_cache["expires_at"] = 0

def _token_body() -> dict:

    # 凭据走 POST body，不进 URL：httpx 的异常文本会把整条 URL 拼进去，
    # 而日志里只要出现过一次这种异常，AppSecret 就出去了。
    return {
        "grant_type": "client_credential",
        "appid": settings.WECHAT_APP_ID,
        "secret": settings.WECHAT_APP_SECRET,
    }

def _store_token(data: dict) -> str:
    _access_token_cache["token"] = data["access_token"]
    _access_token_cache["expires_at"] = time.time() + data.get("expires_in", 7200)
    logger.info("微信 access_token 已刷新")
    return data["access_token"]

async def get_access_token() -> str:
    """获取微信 access_token（稳定版接口），带内存缓存（提前 5 分钟刷新）。"""
    now = time.time()
    if _access_token_cache["token"] and _access_token_cache["expires_at"] > now + 300:
        return _access_token_cache["token"]
```

```
    async with httpx.AsyncClient(timeout=10) as client:
        resp = await client.post(WX_STABLE_TOKEN_URL, json=_token_body())
        if resp.status_code >= 400:
            logger.error("获取 access_token 失败：HTTP %s，响应=%s", resp.status_code,
resp.text[:200])
            raise UserError("微信接口暂不可用，请稍后重试")
        data = resp.json()

    if "access_token" not in data:
        logger.error("获取 access_token 被拒：%s", data)
        raise UserError("微信接口暂不可用，请稍后重试")

    return _store_token(data)

def _access_token_sync() -> str:
    """同步版取 token，和上面共用同一份缓存。

    为什么再来一份而不是复用异步版：笔记写入和创建分享这几条路由是同步 def（跑在
    threadpool 里），在里面 `asyncio.run` 一个协程去拿 token 是给自己埋雷。两边都只
    读写同一个 dict，最坏是并发时多取一次 token，不会拿到错值。
    """
    now = time.time()
    if _access_token_cache["token"] and _access_token_cache["expires_at"] > now + 300:
        return _access_token_cache["token"]

    with httpx.Client(timeout=10) as client:
        resp = client.post(WX_STABLE_TOKEN_URL, json=_token_body())
        if resp.status_code >= 400:
            logger.error("获取 access_token 失败：HTTP %s，响应=%s", resp.status_code,
resp.text[:200])
            raise UserError("微信接口暂不可用，请稍后重试")
        data = resp.json()


    if "access_token" not in data:
        logger.error("获取 access_token 被拒：%s", data)
        raise UserError("微信接口暂不可用，请稍后重试")

    return _store_token(data)

def check_text(openid: str, content: str) -> str:
    """送一段文本给微信内容安全，返回 pass / review / risky / unavailable。

    `unavailable` 是"这次没检成"而不是"内容没问题"：调用方必须把它和 pass 区分开，
    否则接口挂了会导致自检误报绿灯。
    """
    if not settings.SEC_CHECK_ENABLED:
        # 只有测试环境会走到这里（conftest 显式置 false，为的是整套用例不出网）
        return "unavailable"
    if not content or not content.strip():
        return "pass"

    cache_key = (openid, hashlib.sha256(content.encode("utf-8")).hexdigest())
    if _sec_pass_cache.get(cache_key):
        # 同一个人同一段文本判过"正常"就不再问微信。省下来的都是真额度：
        # 一次保存原本要按 6 个字段各打一次，而移动端保存往往把没改过的字段一起 PUT 回来。
        _sec_pass_cache.move_to_end(cache_key)
        return "pass"

    verdict = _ask_sec_check(openid, content)["verdict"]
    if verdict == "pass":
        _sec_pass_cache[cache_key] = True
        _sec_pass_cache.move_to_end(cache_key)
        while len(_sec_pass_cache) > _SEC_PASS_CACHE_MAX:
```

```
            _sec_pass_cache.popitem(last=False)
    return verdict

def probe_sec_check(openid: str, content: str) -> dict:
    """部署自检用的那一趟：必定真打，并且把微信原样回的 errcode 一起交出来。

    为什么不复用 check_text：同样是"没检成"，三种原因的处置完全不同——网络抖动不用管；
    额度打光是今天剩下的时间内容安全等于没做，明天 00:00 自己恢复，不该拦下这次部署；
    凭据或 openid 错则是永远不好，必须拦。自检要按这个分流，就只有拿得到 errcode 才行。
    也不吃判定缓存：自检要的就是这一趟真的出网。
    """
    if not settings.SEC_CHECK_ENABLED:
        return {"verdict": "unavailable", "errcode": "disabled"}
    return _ask_sec_check(openid, content)

def _ask_sec_check(openid: str, content: str) -> dict:
    """真打一趟 msgSecCheck，回 {"verdict": ..., "errcode": ...}。"""
    try:
        payload = {"content": content, "version": 2, "scene": 1, "openid": openid}

        data = None
        for attempt in (1, 2):
            token = _access_token_sync()
            # 不能用 client.post(json=...)：httpx 默认 ensure_ascii=True，中文会被转义成
            # \uXXXX，而微信这个接口**不解析转义**——实测同一段赌博引流文本，原样 UTF-8 体
            # 判 risky(20006)，转义体判 pass。用 json= 就等于把内容安全静默关掉。
            with httpx.Client(timeout=10) as client:
                resp = client.post(
                    f"{WX_SEC_CHECK_URL}?access_token={token}",
                    content=json.dumps(payload, ensure_ascii=False).encode("utf-8"),
                    headers={"Content-Type": "application/json"},
                )
            data = resp.json()
            # 40001 这一类是"这一份 token 不算数了"，不是配置错：清掉缓存再要一次，
            # 只重试一趟，避免凭据真的错了的时候来回打。
            if data.get("errcode") in TOKEN_ERRORS and attempt == 1:
                _invalidate_access_token()
                logger.warning("内容安全拿到 %s（token 已失效），清缓存重试一趟", data.get("errcode"))
                continue
            break
    except Exception as exc:
        # httpx 异常文本可能带整条含 token 的 URL，只进日志
        logger.error("内容安全接口异常 %s: %s", type(exc).__name__, exc)
        return {"verdict": "unavailable", "errcode": "exception"}

    errcode = data.get("errcode")
    if errcode != 0:
        # 实测 44004=空内容（上面已挡）、40003=openid 非法、45009=限流、-1=系统繁忙。
        # 这些都不是"内容有问题"，日志级别按会不会让机制静默失效来分。
        if errcode in (45009, 44991):
            # 额度打光不是"抖一下"：未上架的小程序这个接口只有 100 次/天（2026-09-23 实测
            # 打到过上限），此后今天所有检查看作没做。必须留 ERROR，否则它和一次普通的
            # 网络抖动在日志里长得一模一样。
            logger.error(
                "内容安全额度已打光（errcode=%s），此后今天的检查看作没做。"
                "未上架小程序上限 100 次/天，上架后 200 万次/天：%s",
                errcode, data.get("errmsg"),
            )
        elif errcode in (40001, 42001, 40003, 48001):
            logger.error("内容安全接口配置异常 errcode=%s errmsg=%s", errcode, data.get("errmsg"))
        else:
            logger.warning("内容安全接口暂不可用 errcode=%s errmsg=%s", errcode, data.get("errmsg"))
        return {"verdict": "unavailable", "errcode": errcode}
```

```
    suggest = ((data.get("result") or {}).get("suggest") or "").lower()
    if suggest in ("risky", "review"):
        logger.info("内容安全判定 %s label=%s", suggest, (data.get("result") or {}).get("label"))
        return {"verdict": suggest, "errcode": 0}
    return {"verdict": "pass", "errcode": 0}


def enforce_text_safety(openid: str, *parts) -> None:
    """用户要公开或自己写的内容，送检；命中 risky/review 直接拒。

    策略上两条分开：**内容确实违规 → 拒**；**接口没检成（unavailable）→ 放行并记日志**，
    不能让微信侧抖动变成"用户存不了自己的笔记"。静默失效的风险由部署自检兜（deploy.sh
    会拿一段已知违规的文本打一次，断言必须被拦下来）。
    """
    chunks = []
    for part in parts:
        if isinstance(part, (list, tuple)):
            part = ' / '.join(str(x) for x in part)
        text = str(part or '').strip()
        if not text:
            continue
        window = SEC_CHUNK * SEC_MAX_CHUNKS
        if len(text) > window:
            logger.warning(
                "内容安全只检了前 %d 字，后面 %d 字未送检（openid=%s）",
                window, len(text) - window, openid,
            )
        for i in range(0, min(len(text), window), SEC_CHUNK):
            chunks.append(text[i:i + SEC_CHUNK])

    for chunk in chunks:
        verdict = check_text(openid, chunk)
        if verdict == 'risky':
            raise UserError('内容包含违规信息，无法保存，请修改后再试')
        if verdict == 'review':
            raise UserError('内容需人工复核，请调整表述后再试')

async def get_qr_code_image(scene: str, page: str = "") -> bytes:
    """调用微信 getUnlimitedQRCode 接口，返回图片字节；同一张码只真打一次。

    实测微信回的是 JPEG，调用方别按 PNG 声明硬编码。
    scene 最长 32 字符。page 那个路径必须存在于码指向的那个版本里：不传 env_version
    时微信按正式版（release）出码，所以在过审之前那张码扫开必然打不开——这就是
    SHARE_QR_ENV_VERSION 存在的理由，测试期填 trial，发布前改回 release。

    命中缓存时连 access_token 都不取：这个接口没有鉴权，被反复打的时候烧的是我们
    自己的微信接口配额，而失败方从我们的日志里看不出来（微信回得一切正常）。
    """
    # 填错值微信回 40097 invalid args，用户只会看到一句"小程序码生成失败"，查不到原因。
    # 所以这里宁可静默退回最安全的 release。
    env_version = settings.SHARE_QR_ENV_VERSION
    if env_version not in ("release", "trial", "develop"):
        logger.warning("SHARE_QR_ENV_VERSION=%r 不是合法值，按 release 出码", env_version)
        env_version = "release"

    key = (env_version, page, scene)
    cached = _qr_cache.get(key)

    if cached is not None:
        _qr_cache.move_to_end(key)
        return cached

    body = {"scene": scene, "check_path": False, "width": 280, "env_version": env_version}
    if page:
```

```
        body["page"] = page

    for attempt in (1, 2):
        token = await get_access_token()
        async with httpx.AsyncClient(timeout=15) as client:
            resp = await client.post(
                f"{WX_QRCODE_URL}?access_token={token}",
                json=body,
            )

        # Check HTTP status first
        if resp.status_code != 200:
            raise RuntimeError(f"生成小程序码失败: HTTP {resp.status_code}")

        content_type = resp.headers.get("content-type", "")

        # Handle JSON error responses
        if content_type.startswith("application/json"):
            data = resp.json()
            # 40001 这一类是 token 这一环失效（别的进程轮转过凭据也会造成），
            # 清掉缓存重来一趟；配置真错了第二趟还是同样的错，就照原样抛。
            if data.get("errcode") in TOKEN_ERRORS and attempt == 1:
                _invalidate_access_token()
                logger.warning("小程序码拿到 %s（token 已失效），清缓存重试一趟", data.get("errcode"))
                continue
            raise RuntimeError(f"生成小程序码失败: errcode={data.get('errcode')},
errmsg={data.get('errmsg')}")

        # Verify we got an image
        if "image" not in content_type and content_type:
            raise RuntimeError(f"生成小程序码失败: 非图片响应 (content-type: {content_type})")

        image = resp.content
        _qr_cache[key] = image
        _qr_cache.move_to_end(key)
        while len(_qr_cache) > _QR_CACHE_MAX:
            _qr_cache.popitem(last=False)
        # 这一行是"有没有真的回源"唯一的外部证据：命中缓存时不打。
        # 只记 scene 的前 6 位和字节数——URL 上挂着 access_token，整条请求日志都不能留。
        logger.info("小程序码回源微信 env=%s scene=%s… %d 字节",
                    env_version, scene[:6], len(image))
        return image

# ===== backend/app/services/queue.py — 任务队列 =====
import json

import logging

import redis
from rq import Queue

from app.core.config import settings

logger = logging.getLogger(__name__)

TASK_TTL = 3600

def get_redis_conn() -> redis.Redis:
    return redis.from_url(settings.REDIS_URL)

def get_queue() -> Queue:
    return Queue(connection=get_redis_conn())

def get_task_status(task_id: str) -> dict | None:
```

```
    conn = get_redis_conn()
    data = conn.get(f"task:{task_id}")
    if data:
        return json.loads(data)
    return None

def set_task_status(
    task_id: str,
    status: str,
    result: dict | None = None,
    user_id: str | None = None,
):
    conn = get_redis_conn()
    key = f"task:{task_id}"
    payload: dict = {}
    existing = conn.get(key)
    if existing:
        try:
            payload = json.loads(existing)
        except (TypeError, ValueError):
            payload = {}
    payload["status"] = status
    if result is not None:
        payload["result"] = result
    if user_id is not None:
        payload["user_id"] = user_id
    conn.set(key, json.dumps(payload), ex=TASK_TTL)

# ===== backend/app/tasks/ingest_tasks.py — 内容采集任务 =====
import asyncio
import logging


from app.core.errors import UserError
from app.db.database import SessionLocal
from app.models.note import Note
from app.models.user import User
from app.services import quota
from app.services.ocr import ocr_images, title_hint
from app.services.scraper import scrape_url
from app.services.llm import extract_knowledge
from app.services.queue import set_task_status

logger = logging.getLogger(__name__)

# 任务失败时这个字段会被前端直接塞进 toast，所以宁可笼统，也不要英文栈/onnxruntime/依赖名
GENERIC_TASK_ERROR = "处理失败，请重新提交试试"

def failure_message(exc: Exception) -> str:
    """只有 UserError 的文案是写给用户的；其余异常原文只进日志。"""
    return str(exc) if isinstance(exc, UserError) else GENERIC_TASK_ERROR

def credit_first_note(db, user_id: str, note: Note) -> None:
    """worker 这边也是"写下笔记"的一条路，邀请奖励同样要在这一条上结。

    到账失败不该让整条笔记算处理失败——笔记已经落库了，用户看到的必须是成功。
    """
    try:
        user = db.get(User, int(user_id))
        if user is not None:
            quota.credit_first_note(note, db, user)
    except Exception:
        logger.exception("邀请奖励到账失败（笔记已保存）：user=%s", user_id)
```

```
def _load_task_user(db, task_id: str, user_id: str, generation):
    """把队列里的 (user_id, generation) 换回一个"确实还是提交任务那个人"的 User。

    generation 是这里的关键。SQLite 的 INTEGER PRIMARY KEY 会复用已删除行的 id，
    所以"这个 id 上有人"不等于"还是原来那个人"：A 提交任务后注销、B 注册拿到同一个
    id，少了这层校验，A 抓回来的网页内容就会出现在 B 的笔记列表里。

    返回 None 表示任务就地终止，状态已经写好。generation 为 None 是部署切换期里
    旧代码入队的存量 job——那批 job 拿不到代数，只能退回"人还在就写"。
    """
    user = db.get(User, int(user_id))
    if user is None:
        logger.warning("用户已注销，放弃保存笔记：user_id=%s task=%s", user_id, task_id)
        set_task_status(task_id, "failed", {"error": "账号已注销，无法保存笔记"})
        return None
    if generation is None:
        logger.warning("存量 job 未带账号代数，跳过串号校验：user_id=%s task=%s", user_id, task_id)
        return user
    if user.generation != generation:

        logger.warning(
            "user_id=%s 已被新账号复用（队列 gen=%s，库里 gen=%s），放弃保存笔记：task=%s",
            user_id, generation, user.generation, task_id,
        )
        set_task_status(task_id, "failed", {"error": "账号已变更，无法保存笔记"})
        return None
    return user

def process_url_task(task_id: str, user_id: str, url: str, generation: int | None = None):
    """RQ worker 同步任务：抓取 URL → LLM 提取 → 写入数据库。"""
    try:
        set_task_status(task_id, "processing")

        scraped = asyncio.run(scrape_url(url))
        text = scraped["content"]
        if not text.strip():
            set_task_status(task_id, "failed", {"error": "未能从 URL 提取到有效内容"})
            return

        knowledge = asyncio.run(extract_knowledge(text, fallback_title=scraped["title"]))

        db = SessionLocal()
        try:
            user = _load_task_user(db, task_id, user_id, generation)
            if user is None:
                return
            note = Note(
                user_id=user_id,
                title=knowledge["title"],
                summary=knowledge["summary"],
                key_points=knowledge["key_points"],
                tags=knowledge["tags"],
                original_content=text[:50000],
                source_type=scraped["source_type"],
                source_url=url,
            )
            db.add(note)
            db.commit()
            db.refresh(note)
            credit_first_note(db, user_id, note)
            set_task_status(
                task_id,
                "completed",
                {"note_id": note.id, "title": note.title, "degraded":
```

```
bool(knowledge.get("degraded"))},
            )
        finally:
            db.close()

    except Exception as e:
        logger.exception("URL 任务失败: %s: %s", type(e).__name__, e)

        set_task_status(task_id, "failed", {"error": failure_message(e)})

def process_screenshots_task(task_id: str, user_id: str, images_data: list[bytes], generation:
int | None = None):
    """RQ worker 同步任务：OCR → LLM 提取 → 写入数据库。"""
    try:
        set_task_status(task_id, "processing")

        ocr_text = asyncio.run(ocr_images(images_data))
        if not ocr_text.strip():
            set_task_status(task_id, "failed", {"error": "OCR 未识别到文字内容"})
            return

        knowledge = asyncio.run(extract_knowledge(ocr_text, fallback_title=title_hint(ocr_text)))

        db = SessionLocal()
        try:
            user = _load_task_user(db, task_id, user_id, generation)
            if user is None:
                return
            note = Note(
                user_id=user_id,
                title=knowledge["title"],
                summary=knowledge["summary"],
                key_points=knowledge["key_points"],
                tags=knowledge["tags"],
                original_content=ocr_text[:50000],
                source_type="screenshot",
            )
            db.add(note)
            db.commit()
            db.refresh(note)
            credit_first_note(db, user_id, note)
            set_task_status(
                task_id,
                "completed",
                {"note_id": note.id, "title": note.title, "degraded":
bool(knowledge.get("degraded"))},
            )
        finally:
            db.close()

    except Exception as e:
        logger.exception("截图任务失败: %s: %s", type(e).__name__, e)
        set_task_status(task_id, "failed", {"error": failure_message(e)})

def process_manual_text_task(task_id: str, user_id: str, title: str, content: str,
                             category_id: int | None = None, generation: int | None = None,
                             translate: bool = False):
    """RQ worker 同步任务：手打的原文 → LLM 提炼 → 写入数据库。

    和 URL / 截图两条只差两件事：① 标题用用户自己打的那个，不用模型另起的

    （他要的是"我给标题和原文，模型补提要"——屏幕上那一格叫「摘要」，字段是 summary，
    模型那份只当 fallback）；
    ② 归类是他当场挑的，所以要落 category_id。
```

```
    正文落 original_content、摘要是 summary —— 与另外两条同一份形状，详情页那三块
    （摘要 / 要点 / 原文）才不用为这一种来源特判。

    `translate` 排在这个签名的最后一位：RQ 的 job 带的是位置参数列表，队列里还躺着
    上一版入队的任务（不带这一位）。放中间会让老 job 的 generation 读到 translate 的
    位置、这一位反而空着——A6 那次串号就是这么来的。放最后，老 job 只会拿到默认值 False。
    """
    try:
        set_task_status(task_id, "processing")

        knowledge = asyncio.run(extract_knowledge(content, fallback_title=title,
translate=translate))

        db = SessionLocal()
        try:
            user = _load_task_user(db, task_id, user_id, generation)
            if user is None:
                return
            note = Note(
                user_id=user_id,
                title=title,
                summary=knowledge["summary"],
                key_points=knowledge["key_points"],
                tags=knowledge["tags"],
                original_content=content[:50000],
                source_type="manual",
                category_id=_still_yours_category(db, user_id, category_id),
            )
            db.add(note)
            db.commit()
            db.refresh(note)
            credit_first_note(db, user_id, note)
            set_task_status(
                task_id,
                "completed",
                {"note_id": note.id, "title": note.title, "degraded":
bool(knowledge.get("degraded"))},
            )
        finally:
            db.close()

    except Exception as e:
        logger.exception("手打提炼任务失败: %s: %s", type(e).__name__, e)
        set_task_status(task_id, "failed", {"error": failure_message(e)})

def _still_yours_category(db, user_id: str, category_id: int | None) -> int | None:
    """挑分类到落库中间隔着十几秒，那一格可能已被删掉。

    归属在入队前已经查过一遍，这里只防"删了分类结果整篇笔记失败"——分类没了就当未分类，

    不拿这个去报废一次提炼。
    """
    if category_id is None:
        return None
    from app.models.category import Category

    found = (
        db.query(Category.id)
        .filter(Category.id == category_id, Category.user_id == str(user_id))
        .first()
    )
    return category_id if found else None

# ===== backend/worker.py — 工作进程入口 =====
```

```
"""RQ Worker 入口：python3 worker.py"""
import redis
from rq import Worker, Queue

from app.core.config import settings

if __name__ == "__main__":
    conn = redis.from_url(settings.REDIS_URL)
    worker = Worker([Queue("default", connection=conn)], connection=conn)
    worker.work()

// ===== cloudfunctions/extract/index.js — 云函数-内容提取 =====
const cloud = require('wx-server-sdk')

cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })

// 契约：文本进 → 4 字段 JSON 出。不碰数据库、不认身份、不含业务分支。
// 提示词**只由调用方持有**并通过 event.system_prompt 传进来：这里不留第二份，
// 否则改了一处忘另一处，线上就会出现"后端以为换了 prompt、模型其实没换"。
// 失败时只回 {error: "..."}，让后端区分「立刻降级」与「值得重试」。

// 模型 id 留成环境变量，改它不必重传 zip。**默认值是 2026-09-20 实测出来的，不是文档抄来的**：
// 控制台「AI 资源 → 生文模型」里带「免费额度」标记的 hy3 与 hy3-preview 行为完全不同——
// hy3 一律 0.68s 内抛 HTTP 429 `AI_MODEL_NOT_ENABLED`（"To use HY3, please switch to the resource
pack",
// 即要求先切换为资源点套餐）；hy3-preview 则正常返回 4 字段并带 usage，实测 1.27-2.87s。
// 所以**不切计费也能用上成长计划的免费额度，走的是 hy3-preview 这一行**。别改回 hy3。
const MODEL_ID = process.env.HUNYUAN_MODEL_ID || 'hy3-preview'
const MAX_INPUT = 12000

function stripFences(s) {
  const t = (s || '').trim()
  if (!t.startsWith('```')) return t
  return t
    .split('\n')
    .filter((l) => !l.startsWith('```'))
    .join('\n')

    .trim()
}

// 与后端 _parse_response 保持同一套宽容度：模型偶尔会吐非 JSON，此时宁可回原文片段，
// 也不要让整条链路失败。
function normalize(parsed, fallbackTitle) {
  const title =
    typeof parsed.title === 'string' && parsed.title.trim()
      ? parsed.title.trim().slice(0, 500)
      : (fallbackTitle || '未命名笔记').trim().slice(0, 500)
  const summary = typeof parsed.summary === 'string' ? parsed.summary : ''
  const keyPoints = Array.isArray(parsed.key_points)
    ? parsed.key_points.filter((v) => typeof v === 'string' || typeof v === 'number').map(String)
    : []
  const tags = Array.isArray(parsed.tags)
    ? parsed.tags.filter((v) => typeof v === 'string' || typeof v ===
'number').map(String).slice(0, 20)
    : []
  return { title, summary, key_points: keyPoints, tags }
}

exports.main = async (event) => {
  const text = typeof event === 'string' ? event : (event && event.text) || ''
  const fallbackTitle = (event && event.fallback_title) || ''
  const systemPrompt = (event && event.system_prompt) || ''
```

```
if (!text.trim()) return { error: 'empty input' }
if (!systemPrompt.trim()) return { error: 'missing system_prompt' }

let userContent = text.length > MAX_INPUT ? text.slice(0, MAX_INPUT) : text
if (fallbackTitle) userContent = `参考标题：${fallbackTitle}\n\n${userContent}`

let result
try {
  const ai = cloud.ai()
  const model = ai.createModel('cloudbase')
  result = await model.generateText({
    model: MODEL_ID,
    messages: [
      { role: 'system', content: systemPrompt },
      { role: 'user', content: userContent },
    ],
    temperature: 0.3,
  })
} catch (e) {
  // 抛出来比返回 500 好定位：网关会把未捕获异常压成一个看不出原因的调用失败。
  // 但光有 e.message 不够——axios 的 message 只有 "Request failed with status code 429"，
  // 真正的拒绝原因在服务端返回体里，而 429 到底是"并发超限"还是"额度不可用"决定了下一步做法。
  //
  // retryable 这个字段是给后端的：模型侧的 429/5xx 是**间歇性的**（实测空闲后的第一次调用
  // 必吃一个 429、紧接着的几次全好），而后端默认把"函数返回 error"当成不可重试错误直接降级，

  // 结果就是每条空闲后的第一条笔记静默丢掉摘要。所以这里把可重试性显式带出去。
  const resp = e && e.response
  if (resp) {
    let body = resp.data
    if (typeof body !== 'string') {
      try {
        body = JSON.stringify(body)
      } catch (stringifyError) {
        body = String(resp.data)
      }
    }
    return {
      error: `cloud.ai 返回 HTTP ${resp.status}: ${(body || '').slice(0, 500)}`,
      retryable: resp.status === 429 || resp.status >= 500,
    }
  }
  return {
    error: `cloud.ai 调用异常: ${e && e.message ? e.message : String(e)}`,
    retryable: true,
  }
}

if (result && result.error) {
  return { error: `模型返回错误: ${JSON.stringify(result.error)}` }
}

const content = stripFences(result && result.text)
if (!content) return { error: '模型返回空内容' }

let parsed
try {
  parsed = JSON.parse(content)
} catch (e) {
  return {
    title: (fallbackTitle || '未命名笔记').trim().slice(0, 500),
    summary: content.slice(0, 500),
    key_points: [],
    tags: [],
```

```
    }
  }

  if (!parsed || typeof parsed !== 'object' || Array.isArray(parsed)) {
    return { error: '模型返回的不是 JSON 对象' }
  }

  const out = normalize(parsed, fallbackTitle)
  out.usage = result.usage || null
  return out
}
```